

Early Stopping for Any Number of Corruptions

Julian Loss^{1,*} and Jesper Buus Nielsen^{2,**}

¹ CISA Helmholtz Center for Information Security, Germany

² Aarhus University, Denmark

Abstract. Minimizing the round complexity of byzantine broadcast is a fundamental question in distributed computing and cryptography. In this work, we present the first *early stopping* byzantine broadcast protocol that tolerates up to $t = n - 1$ malicious corruptions and terminates in $\mathcal{O}(\min\{f^2, t\})$ rounds for any execution with $f \leq t$ *actual corruptions*. Our protocol is deterministic, adaptively secure, and works assuming a plain public key infrastructure. Prior early-stopping protocols all either require honest majority or tolerate only up to $t = (1 - \epsilon)n$ malicious corruptions while requiring either trusted setup or strong number theoretic hardness assumptions. As our key contribution, we show a novel tool called a *polariser* that allows us to transfer certificate-based strategies from the honest majority setting to settings with a dishonest majority.

1 Introduction

In the problem of byzantine broadcast [22], a sender P_s holds a value v that it wants to share among n parties $P_1, \dots, P_n$ using a distributed protocol Π with the following properties: 1) *validity*: if the sender is honest (i.e., it follows the protocol description of Π correctly), all honest parties output v 2) *agreement*: all honest parties output the same value v' from Π . Broadcast is an integral building block in many cryptographic and distributed protocols, e.g., multi-party computation, verifiable secret sharing, and state-machine replication. One of the most important efficiency metrics for a broadcast protocol is its *round complexity*: how many rounds does the protocol run for until all parties have terminated?

A seminal result of Dolev and Strong [13] shows that any broadcast protocol tolerating $t < n$ malicious parties runs for at least $t + 1$ rounds in some runs. In the same work, they also give a protocol that shows the tightness of their bound. However, their bound is known to be loose for protocol executions where the number of corruptions f is less than t , i.e., $f < t$, and where eventual agreement is allowed. A seminal result of Dolev and Strong [13] shows that any deterministic broadcast protocol tolerating $t < n$ malicious parties runs for at least $t + 1$ rounds. In the same work, they also give a protocol that shows the tightness of their bound. However, their bound is known to be loose for protocol executions where the number of corruptions f is less than t , i.e., $f < t$. In this case, the tightest lower bound [12] says that any (deterministic) protocol must run for at least $f + 2$ rounds. Intrigued by this vexing gap, a long line of work has studied so-called *early stopping* protocols which terminate in $O(g(f)) = o(t)$ rounds (for some function g) in any execution where the actual number of corrupted parties f is sufficiently small compared to t .

* This work is funded by the European Union, ERC-2023-STG, Project ID: 101116713. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

** Funded by the Danish Independent Research Council under Grant-ID DFF-3103-00077B (CryptoDigi).

Early stopping protocols are known for both the information theoretic setting with $t < n/3$ malicious corruptions [7, 18, 2, 23] and the authenticated setting with $t < n/2$ corruptions [28]. To the best of our knowledge, however, little is known about early stopping protocols for the setting of $n/2 \leq t < n$ malicious corruptions. On one hand, several randomized protocols achieve sublinear (in n) round complexity for broadcast in the dishonest majority setting [17, 16, 8, 31, 30]. However, these protocols require the maximum number t of corruptions to be at most a constant fraction of n in order to stop early (some require t to be much smaller).

All in all, for the full corruption threshold $t = n - 1$, the tightest lower bound says that any protocol must run for at least $f + 2$ rounds, whereas the best protocol uses n rounds in all runs, even for low f . Clearly, there is a fundamental gap in our understanding of early stopping protocols when $t = n - 1$. Motivated by this discussion, we pose the following question: *Are there early stopping broadcast protocols tolerating up to $t = n - 1$ corruptions?*

1.1 Our Contribution

In this work, we answer this question in the affirmative by providing the first early stopping protocol CDC for arbitrary majority corruption, i.e., $t = n - 1$. Concretely, its properties can be summarized as follows:

- CDC tolerates any number $t < n$ of malicious corruptions.
- For any execution with $f \leq \sqrt{t}$ faults, CDC terminates in $\mathcal{O}(f^2)$ rounds. It always terminates within $\mathcal{O}(t)$ rounds. Prior work achieves either $t + 1$ rounds deterministically or requires that the maximum number of faults satisfy $t = (1 - \epsilon) \cdot n$ for some $\epsilon > 0$ in order to achieve early stopping.
- Our protocol is secure with respect to a *strongly adaptive adversary*. This type of adversary can observe an honest party’s messages, corrupt it, and replace its messages with its own before these messages are delivered. This sets our work apart from existing early stopping protocols for the dishonest majority regime with a strongly adaptive adversary. Namely, existing protocols require either 1) strong number theoretic hardness assumptions (i.e., time-locked puzzles) [30, 29] or 2) tolerate only $t = n/2 + O(1)$ malicious corruptions [17, 16] in order to stop early.
- CDC is deterministic and works in the plain public key model. By comparison, existing early stopping protocols for majority corruption are all randomized and, in many cases, require strong setup assumptions.

In summary, CDC is the first early stopping protocol for $t = n - 1$ and makes a significant improvement over the state of the art for any execution with $f = o(\sqrt{n})$ corruptions. We give further comparison with existing literature in the related work section.

1.2 Technical Overview

We now give an overview of our techniques. We begin by giving a brief recap of the classical Dolev-Strong protocol [13] DSC. We then explain the difficulty of making this protocol early stopping and our key insights to overcome it.

Recap: the Dolev-Strong protocol. DSC achieves a round complexity of $t+1$ against a strongly adaptive malicious adversary corrupting up to $t < n$ parties. It works as follows for a sender P_s holding a message m :

- In round 1, P_s signs m and sends it to all parties.
- In any round $i \leq t$, a party P_j does as follows. If it receives a message m together with a list of valid signatures from i distinct parties for the first time, it adds m to a set of accepted messages $\mathcal{A}$. Then, P_i adds its own signature to the list, and forwards it to all parties so that they add it to $\mathcal{A}$ at most one round later.
- In round $t+1$, a party P_i executes the above rule to update $\mathcal{A}$, but does not forward a new list. Instead P_i determines its output as follows. If $\mathcal{A} = \{\}$ or $|\mathcal{A}| > 1$, output a default output NoMsg. If $\mathcal{A} = \{m\}$, output m .

Clearly, if an honest P_i adds m to $\mathcal{A}$ at any point during the first t rounds of the protocol, all honest parties add it to $\mathcal{A}$ at most one round later. If P_i adds m to $\mathcal{A}$ in round $t+1$, it sees m with $t+1$ signatures and thus knows that at least one honest party P_j has previously signed m in a previous round. Since P_j was honest in that round, it would have added its own signature to the list of t signatures it needed to add m to $\mathcal{A}$ and passed on the resulting list of $t+1$ signatures to all parties. Hence, all honest parties add m to $\mathcal{A}$ by round $t+1$.

Why making DSC early stopping is hard. Stopping DSC early turns out to be very challenging. Surprisingly, however, this does not result from a fully malicious sender P_s that sends conflicting (signed) messages in DSC to break consistency, as this allows all honest parties to detect and prove that P_s is acting dishonestly. The central difficulty arises already from crash faults: P_s can simply not send any message to (some of) the parties. Note that if P_s sends no signature then DSC runs in silence for $t+1$ rounds before outputting NoMsg. We would like to terminate this earlier. However, the sender might send a signature only to a single honest party in round 1. Or send it to no honest party but collude with some P_i forwarding the signature from P_s to a single honest party in round 2, *et cetera*. To detect this and stop early it would help if honest parties from round 1 could prove that they did not get a message from P_s . But how to prove this? This is a comparatively simple task in the honest majority setting where there are $n-t \geq t+1$ honest parties. At a high-level, parties can simply collect a certificate of accusations against the sender for not sending them a message. If they can obtain $t+1$ signed accusations, they can disqualify the sender and stop the protocol. On the other hand, if P_s cannot be disqualified, then at least one honest party must have received a message from P_s and can forward it to all other parties. Unfortunately, this strategy fails when $t \geq n/2$, as there are only $n-t < t+1$ honest parties so a certificate might not be constructible. Thus, the obstacle we must overcome is to design an analogue of a certificate for the dishonest majority setting.

Polarisers to the rescue. Our key tool for achieving this is a novel primitive called a *polariser*. Informally, a polariser partitions the parties into two ‘polarized’ sets **Alive** and **Corrupt**. Polarisers are updated continuously throughout the protocol and maintain the following properties. First, an honest party P_i accepts a polariser from another party (and subsequently

updates its own polariser) only if it itself is in **Alive**. Moreover, for any party $P \in \text{Corrupt}$, a polariser contains accusation against P from *all* parties in **Alive**. And it is ensured that honest parties never accuse honest parties. As it turns out, these properties make it possible for an honest party P_i to justify any decision in our protocol and convince other honest parties to take the same decision. A crucial observation is that it follows from the properties that all honest parties are in **Alive** and therefore P_i knows it can forward the polariser and have it accepted by other *honest* parties: they too are in **Alive**. Thus, polarisers act as our replacement for certificates. The idea behind creating a polariser is surprisingly simple. As an example, suppose the sender P_s does not send a party P_i a message in the first round. Now, P_i can publicly accuse P_s . To deal with having accusations levelled against honest senders, note that honest parties will move P_s to **Corrupt** only if it was accused by all parties in **Alive**. Since P_i never accuses an honest sender P_s and is itself in **Alive**, this precludes honest parties from moving an honest P_s to **Corrupt**. However, this creates a different problem: if P_j is also dishonest, but is itself in **Alive**, it can simply not accuse P_s . In this case, P_s cannot be moved from **Alive** to **Corrupt**, since not all parties in **Alive** have accused P_s . To deal with this type of behaviour from dishonest parties, we present a recursive solution for generating a polariser. In our running example from above, we require that either P_j sends whatever it got from P_s or it accuses P_s of cheating. If P_i receives neither of those two things from P_j , P_i knows that P_j is itself corrupt and accordingly accuses P_j . And, importantly, P_i expects every other party P_k to accuse P_j too, or send to P_i the reason that it did not accuse P_j , namely an accusation of P_j against P_s (which would resolve the issue). If P_k does not send an accusation or resolvment, then P_i will accuse P_k , *et cetera*. In each recursive step one more corrupt party is accused, and there are at most f corrupt parties, so the recursion stops in at most f steps. When it stops, then in the view of every honest party either P_s can be moved from **Alive** to **Corrupt** or a signature from P_s was received. Hence each honest party receives either a signed message from P_s or a polariser showing that P_s is corrupt.

Justifying outputs and graded broadcast. Polarisers can be used to justify the output (or non-output) of any subprotocol to other parties. To do so, parties will either send their entire view of the protocol transcript so far in case a subprotocol produces output, or send a polariser to justify not having output. We show a protocol **GSTM** for *graded broadcast* that is inspired by the protocol of Koo et al. [17]. Roughly, **GSTM** outputs a message m together with a grade $g \in \{0, 1, 2\}$ and a proof π that justifies the combination of m and g . The grade g reflects a party's confidence in its output. Grade $g = 2$ indicates high confidence, meaning that m should be output, and all other parties have grade at least 1 for the same message m . Grade $g = 1$ indicates low confidence in m , meaning that some parties might not have received the message m . Finally, $g = 0$ indicates that the message was not received, the sender was corrupt, and some dummy **NoMsg** should be output. Validity ensures that all parties output $(m, 2)$ whenever the sender is honest. The crucial property of **GSTM**, however, is that if a dishonest party can produce a justified output then the grading rules from above apply too. So a corrupted party basically cannot justify an output unless that output could have been produced by an honest party.

Putting things together: Protocol DC. Using **GSTM** we are able to run a broadcast protocol **DC** that resembles the well-known phase-king approach [6] from the honest majority setting. In this style of protocol, one rotates through $f + 1$ leaders until agreement on an output is detected. Essentially, each leader is instructed to broadcast via **GSTM** whatever it received from the previous instance. The crucial idea is that a malicious leader can never undo the progress that the protocol has made so far by making them choose a different output from one that they have already agreed on (or not choosing the output of an honest sender). This is because each time a new broadcast is initiated by a leader, its input must be justified by the entire view of the protocol so far. Therefore it must use an input which some honest party could have used, or choose to abort the protocol. More precisely, once an output m is received with a positive grade g in the j th leader iteration, parties set $m_j = m$ and broadcast this message once it is their turn to be the leader. (If the previous king aborted then they pick the most recent such value, if there is one, and otherwise they pick a fixed default output). The consistency properties of **GSTM** and the justification that comes along with any new output ensures that a dishonest leader can never introduce a new message once parties have already agreed on a message m . Once parties see an output m with grade 2, they can detect agreement, forward the justification for this output to all parties, and terminate. After $f + 1$ leader rotations, at least one of the leaders will be honest, giving grade 2, at which point the agreement detection is triggered and the protocol terminates.

CDC: Ensuring $O(t)$ round-complexity. In the worst case, each of the leader iterations in **DC** identifies only a single party (i.e., the leader) as malicious. Since each of these steps takes roughly f rounds, we end up with an overall complexity of $O(f^2)$ for **DC**. To avoid exceeding $O(t)$ for the round complexity, we can simply stop the protocol after running for $\ell = O(t)$ rounds. At this point, we can afford to run an additional instance of **DSC** to reach agreement. More precisely, any party that has not terminated by round $\ell - 1$ will thus use **DSC** to broadcast its view of the protocol so far to all parties after completing iteration ℓ . (Running for one more iteration ensures that all parties have time to be forwarded justifications from already terminated parties). Once **DSC** terminates after another $O(t)$ rounds, all parties can locally decide on a correct output which, by the justification properties of **GSTM**, will be consistent with parties who have already terminated. This solution, however, still has an undesirable property: honest parties can terminate $O(t)$ rounds apart, whenever one party terminates in iteration $\ell - 1$, but other parties keep on running. This would make the protocol very difficult to use as a subroutine in higher-level application. To have parties terminate one round apart, our actual protocol **CDC** replaces **DSC** with a new protocol **WES** (**WES** stands for *weak early stopping*) that terminates in $O(f)$ rounds if the sender is honest and $O(t)$ rounds in any other case—furthermore, parties terminate at most one round apart. This allows honest parties who terminate early in **DC** to always broadcast their justified outputs via **WES**. If any **WES** reports a justified output of **DC** that will be the final output, otherwise the final output will be **NoMsg**. Parties can terminate immediately upon receiving output from the first terminating instance of **WES** reporting a justified output of **DC**. If an honest party saw a justified output of **DC** it gets reported in $O(f)$ rounds and the overall protocol terminates in $O(f^2)$ rounds. If no honest party saw a justified output from **DC** in

$O(t)$ rounds, there are (at least) $f = \sqrt{t}$ corruptions and then the protocol is allowed to run for $O(t)$ rounds. We believe that WES may have other natural applications.

Achieving Polynomial Complexity. A final wrinkle is that sending along the protocol's entire transcript in every step would result in exponential communication complexity. However, this turns out to be unnecessary, as P_i can simulate another party P_j 's behaviour from its past messages. This means that P_j need only send information associated with the most recent protocol step every time it is asked to justify one of its outputs. Thus, our combined protocol CDC ends up with a communication complexity of $O(n^4\lambda)$, where λ is the length of a signature.

1.3 Related Work

Below, we summarize some early stopping protocols from the literature. Dolev, Strong, and Reischuk [12] define two notions of agreement: simultaneous agreement (SA) and eventual agreement (EA). In SA, parties output in the same round, whereas in EA, they are allowed to output in different rounds (but must of course still agree on the output itself). They show that for SA, any protocol will have runs with $t + 1$ rounds even when $f = 0$. However, for EA, they show a stronger lower bound of $f + 2$ rounds.³ Moreover, for EA, their work does not give a protocol matching the $f + 2$ lower bound.

Deterministic Protocols. To the best of our knowledge, the first early stopping protocol for byzantine agreement (which implies broadcast for $t < n/2$) with optimal resilience $t < n/3$ in the information theoretic setting was due to Berman et al. [7], who gave a protocol with (optimal) round complexity $\min\{f + 2, t + 1\}$ and exponential communication. Their work builds on earlier work of Berman and Garay [5] who achieved the same round complexity with polynomial complexity and $n > 4t$. Garay and Moses [18, 19] later improved the communication and computation for the corruption-optimal protocol to polynomial. However, their protocol achieves a slightly worse round complexity for the early stopping case of $\min\{f + 5, t + 1\}$. Much later, Abraham and Dolev [2] gave the first protocol with optimal round complexity $\min\{f + 2, t + 1\}$ and polynomial communication/computation. In the authenticated setting with $t < n/2$ and plain PKI, Perry and Toueg [28] showed a protocol with polynomial communication and computation complexity and a round complexity of $\min\{2f + 4, 2t + 2\}$.

Randomized Protocols. In the following, we let δ denote the failure probability of a protocol. There are various randomized protocols for the honest majority setting with constant expected round complexity for both the $t < n/3$ (information theoretic) setting [15, 25] and $t < n/2$ (authenticated) setting [20, 1]. They can all be made to terminate early with worst-case failure probability δ by running them for $O(\log(1/\delta))$ iterations (each iteration has constant many rounds).

³ The proof uses a hybrid argument appealing to agreement $e^{\text{poly}(t)}$ times, so holds only for protocols with agreement error $e^{-\text{poly}(t)}$. The argument appeals to validity only twice so the validity error may be any constant $< 1/2$.

One can use similar ideas to turn expected constant round protocols in the dishonest majority setting with $t < n$ corruptions into protocols that always stop early and have failure probability δ . Here, randomized protocols were first explored by Garay et al., who showed an expected $O(2t - n)^2$ -round protocol from plain PKI for any $t < n$ [17]. Their approach was later improved by Fitzi and Nielsen [16], who showed a protocol with $O(2t - n)$ complexity in the same setting. These protocols lead to early stopping protocols with round complexities $O(\log(1/\delta) + (2t - n)^2)$ and $O(\log(1/\delta) + 2t - n)$, respectively, and failure probability δ . However, since $O(2t - n) = O(n)$ (regardless of f) whenever $t = (1 - \epsilon) \cdot n$ where $\epsilon > 0$, these protocols are not early stopping.

Chan et al. [8] presented a randomized broadcast protocol with trusted setup and tolerating any $(1 - \epsilon)$ -fraction of adaptive corruptions (for an arbitrary constant $\epsilon > 0$) by assuming no after-the-fact removal of messages. Their protocol achieves a round complexity of $O(\log(1/\delta)) \cdot (n/(n - t))$ for failure probability δ . Also assuming trusted setup and no after-the-fact removals, but tolerating up to $t = n - 1$ corruptions, Wan et al. [31] give a protocol achieving expected $O((n/(n - t))^2)$ round complexity and $O(\log(1/\delta)/\log(n/t)) \cdot (n/(n - t))$ complexity for failure probability δ . Protocols tolerating adaptive corruptions with after-the-fact message removals and $t < (1 - \epsilon)n$ corruptions were studied by Wan et al. [31] and more recently by Srinivasan et al. [29], who gave protocols from time-lock-assumptions achieving round complexities of $(n/(n - t))^2 \cdot \text{polylog}(\lambda)$ (for failure probability negligible in λ) and $O(\log(1/\delta)) \cdot (n/(n - t))$, respectively. Most recently, Alexandru et al. [4] showed how to remove the need for trusted setup in order to obtain $O(\log(1/\delta)) \cdot (n/(n - t))$ round complexity. Although these protocols all achieve early stopping (with failure probability δ) for $t = (1 - \epsilon) \cdot n$, they are also not early stopping when $t = n - O(1)$. Namely, in this case, their round complexity is at least $O(n/(n - t)) = O(n)$.

Lower Bounds, communication optimizations, and weaker models. As mentioned above [12] showed that the round complexity of an early stopping algorithm with eventual agreement in an execution with f faults is lower bounded by $\min\{f + 2, t + 1\}$. This was later extended by Keidar and Rajsbaum [21] to the setting of omission faults, which can fail to send or receive some of their messages. They demonstrate that early stopping broadcast/agreement algorithms require the same complexity as in the malicious setting if agreement is required to be *uniform*, i.e., omission faulty parties that output, must output consistently with the honest parties. Chandra et al. present reliable broadcast protocols achieving $f + 2$ rounds in the crash fault model and $2f + 3$ rounds in the omission fault model [9]. (In reliable broadcast, parties need not terminate when the sender is dishonest.) The latter result was later improved to $f + 2$ by Parvédy and Raynal [27] to $\min\{f + 2, t + 1\}$ round complexity and $O(n^2 \cdot f)$ communication complexity in the omission fault model. Finally, Albouy et al. [3] show an optimistically early stopping broadcast protocol with security against any number of corruptions and with polynomial communication. Their protocol achieves $t + 1$ rounds when the sender is malicious and $\max\{2, f + t - n + 3\}$ rounds when the sender is honest.

From the perspective of communication complexity, a result by Dolev and Lenzen [10] shows that any (deterministic) early stopping algorithm with optimal round complexity re-

quires sending $O(nt + t^2f)$ messages. This tightens the famous Dolev-Reischuk bound for the early stopping case [11]. On the other hand, the recent result of Lenzen and Sheikholeslami [23] demonstrate that this bound can be circumvented by presenting an early stopping protocol with (suboptimal) $O(f)$ round complexity but significantly improved $O(nt)$ communication complexity.

2 Preliminaries

We work with directed trees Tree with a single root and edges pointing towards the leafs. For a tree Tree we use $\text{path} \in \text{Tree}$ to denote a path $(r, \dots, l)$ from the root r to a leaf l . We let $\text{depth}(\text{Tree}) = \max_{\text{path} \in \text{Tree}} (|\text{path}| - 1)$. The tree with only a root thus has $\text{depth}(\text{Tree}) = 0$ and if the tree is empty $\text{depth}(\text{Tree}) = -1$. For $\text{path} = (r, \dots, l)$ we let $\text{leaf}(\text{path}) = l$. We assume a synchronous model with n parties $\mathbb{P} = \{P_1, \dots, P_n\}$. The computation proceeds in rounds where in each round each party can send a message to each other parties that is guaranteed to arrive by the end of that round. We assume a rushing adversary that can adaptively corrupt parties and replace or delete any of the messages they sent for a round and which have not yet been delivered. We use t to denote the maximum number of corrupted parties and $f \leq t$ to denote the actual number of corrupted parties. We allow t to take any value $0 \leq t \leq n$.

We assume a PKI. In an initial setup round each party P_i generates a key-pair $(\text{sk}_i, \text{vk}_i)$ for a signature scheme and announces vk_i to a public bulletin board. As is standard for this line of work, we assume the Dolev-Yao model [14] and treat signatures as information theoretic objects with perfect unforgeability. Throughout, we denote the size of a signature in bits as λ .

Definition 1 (Broadcast). *Let Π be a protocol executed by n parties $P_1, \dots, P_n$, where a designated sender P_s holds input x and each party P_i terminates upon giving output y_i . We say that Π is a broadcast protocol if it has these properties:*

- **Validity:** *If P_s is honest, each honest party P_i outputs $y_i = x$.*
- **Agreement:** *For honest parties P_i and P_j , $y_j = y_i$.*

Sequential composition of protocols without simultaneous termination. When describing our protocols we will assume that all parties get input in the same round. If a party has no input in a protocol we assume that they nonetheless get a tacit, dummy input, to ensure they know in which round to start running. We also assume that all sub-protocols give outputs in the same round. This ensures that if the parties call a sub-protocol and then proceed when it gives output, then they are still synchronized. Under these conditions we will design protocol where parties might terminate at most one round apart. This leads to problems with composition: when using a protocol as a subroutine, we assume parties give outputs in the same round. But in many of our protocols, parties terminate one round apart. This can, however, be handled using known techniques for sequential composition of protocols without simultaneous termination at a blowup in round complexity of just 2. Details can be found in [24] and Chapter 7 in [26]. Here, we sketch the main idea for completeness. Protocols which

assume that the parties start in the same round will be compiled into protocols tolerating that they start one round apart. The compiler works as follows. If parties start a protocol Π one communication round apart, then after P_i sends its messages for protocol round r of Π , it will wait for two underlying communication rounds to ensure it received messages from honest parties sending their messages one underlying communication round later than P_i . Then, P_i computes its messages for the next protocol round and sends them out, waits for two communication rounds *et cetera*. This leaves the problem that the parties might terminate two communication rounds apart. This would be a problem for sequential composition as we want them to start the next protocol only one communication round apart. This can be mitigated when Π has justified outputs. When the first party gets an output it sends it to the other parties. The output will arrive within 1 *communication* round. When a party sees a valid incoming protocol output, they adopt this as their own output and forward it to all parties. Now all parties terminate at most one communication round apart. We will call this the *staggering compiler*.

3 Polarisers and Transferable Justifiers

We first put in place a tool which will allow us to write later protocols more concisely. The tool is called a *polariser* as it polarises the n parties into two disjoint sets S and T of which we know all honest parties are fully inside one of the sets. An external party might not know whether S or T contain the honest parties, but an honest party will of course know which set it is in.

Polarisers are used for proving that a corrupted party P_i did *not* send a message. To motivate their design we first discuss this issue. Consider a party P_i which is to send a message m of a particular form to P_k , say it should be signed. We would like to know when this was *not* done and have a *transferable* proof of this. If we have a bound $t < n/2$ on how many corruptions there can be then this is easy. You can ask P_i to send m via all other parties P_j and have all P_j forward m to P_k or a signature $\sigma_j = \text{Sig}_{sk_j}((\text{Acc}, P_i, P_j))$ which is a signed accusation of P_i that P_j did not send a message. Now P_k either gets m or a certificate of $t + 1$ accusations which can act as a transferable proof that P_j did not send a message. In contrast, in the dishonest majority setting, simple majority voting about whether the message was sent will not solve the problem.

The solution is polarisers. The core of a polariser will be a tuple $(\text{Alive}, \text{Corrupt}, \text{Accuse})$, where $\mathbb{P} = \text{Alive} \cup \text{Corrupt}$ and for all parties in **Alive**, we have a signed accusation for each party in **Corrupt**. Assuming that all honest parties send all intended messages and honest parties only accuse parties which fail to send a message, this leaves only two cases when seeing $(\text{Alive}, \text{Corrupt}, \text{Accuse})$. Either all the honest parties are in **Alive** or all the honest parties are in **Corrupt** (if there is an honest party in both **Alive** and **Corrupt** then an honest party accused an honest party). But it might of course be that all the parties in **Alive** are corrupted and falsely accusing the parties in **Corrupt**. This is hard to catch in the dishonest majority case where it can happen that $|\text{Alive}| > |\text{Corrupt}|$ and all parties in **Alive** are corrupt. This prevents external agreement on who is corrupt.

The trick is to give up on externally valid certificates and go for a weaker type of certificate which maintains transferability only within the context of the current protocol. An honest party P_i can check whether $P_i \in \text{Alive}$ or $P_i \in \text{Corrupt}$. If $P_i \in \text{Corrupt}$, then reject the polariser. Note that in this case *all* honest parties are in Corrupt and will therefore also reject the polariser if it is sent to them. If $P_i \in \text{Alive}$, then accept the polariser. Note that in this case *all* honest parties are in Alive and will therefore also accept the polariser.

Definition 2 (Polariser). Let $\text{Pol} = (\text{Alive}, \text{Corrupt}, \text{Accuse})$ be a tuple where we refer to set Alive as the alive parties, to set Corrupt as the corrupt parties, and to set Accuse as the accusations. We define the following structural properties:

- **Justifiability.** For every $P_j \in \text{Corrupt}$ and for every party $P_i \in \text{Alive}$, there exists $A_{i,j} \in \text{Accuse}$, where $A_{i,j}$ denotes a value $(\text{ACC}, P_i, P_j, \sigma_i)$, where $\text{Ver}_{\text{vk}_i}((\text{ACC}, P_i, P_j), \sigma_i) = \top$.
- **Completeness.** $\text{Alive} \cap \text{Corrupt} = \emptyset$ and $\text{Alive} \cup \text{Corrupt} = \mathbb{P}$.

We define the following contextual property:

- **Accusation Soundness.** If P_i and P_j are honest then the adversary cannot construct a valid $A_{i,j}$ in PPT, in particular there is no such $A_{i,j}$ in Accuse .

We call a polariser Pol a P_i -polariser if $P_i \in \text{Pol.Corrupt}$. We use $\overset{i}{\nearrow}$ to denote the set of P_i -polarisers. As with the Landau notation for asymptotic complexity we misuse notation and use $\text{Pol} = \overset{i}{\nearrow}$ to denote that $\text{Pol} \in \overset{i}{\nearrow}$. We also sometimes let $\overset{i}{\nearrow}$ denote a generic element from $\overset{i}{\nearrow}$.

In all our protocols constructing polarisers we only sign messages of the form $(\text{ACC}, P_i, P_j, \sigma_i)$ if P_j is corrupt. Therefore:

Lemma 1 (Polarisation Lemma). Let Honest be the set of honest parties and let Pol be a polariser. Then $\text{Honest} \subset \text{Pol.Alive}$ or $\text{Honest} \subset \text{Pol.Corrupt}$.

Proof. By Completeness, $\text{Honest} \subset \text{Pol.Alive} \cup \text{Pol.Corrupt}$, and it cannot be the case that $P_i \in \text{Pol.Alive}$ and $P_j \in \text{Pol.Corrupt}$ are honest, because then by Justifiability $A_{i,j} \in \text{Accuse}$, contradicting Accusation Soundness. $\square$

3.1 Transferable Justifiers

Our second general tool is the concept of a transferable justifier for a protocol output. Recall that the purpose of polarisers is to get a transferable proof that some party did not send a message. From these, we build increasingly complex messages and eventually a justified output. It is helpful to have some general machinery for talking about transferable justifiers.

Definition 3 (Justifier). We call a PPT predicate $J : \mathbb{P} \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{\top, \perp\}$ a justifier predicate. If for a party P_i , a message m and a proof π we have that $J(P_i, m, \pi) = \top$ then we say that P_i accepts the message m with justifier π . We require that justifiers are transferable, i.e., if P_i and P'_i are honest then $J(P_i, m, \pi) = \top$ implies that $J(P'_i, m, \pi) = \top$. We use $J(m, \pi) = \top$ to denote that $J(P_i, m, \pi) = \top$ for all honest P_i . By transferability this is the implied if it holds for a single honest P_i .

Remark 1 (Justifier predicates do not depend on state, only name). We note that the notation $J(P_i, m, \pi) = \top$ does not mean that J depends on the state of P_i . As indicated by the function signature $J : \mathbb{P} \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{\top, \perp\}$ the predicate only depends on the *name* of the party computing the predicate. This can still be non-trivial. As an example consider a protocol where P_j was to send a message and let $\text{NoMsg}^{(j)}$ be a special symbol denoting that P_j sent no message. Then a justifier predicate for this could be

$$J(P_i, \text{NoMsg}^{(j)}, \text{Pol}) \equiv P_i \in \text{Pol.Honest} \wedge \text{Pol} = \overset{j}{\nrightarrow}, \quad (1)$$

i.e., P_i accepts that P_j sent nothing if Pol proves that P_j is corrupt *and* Pol proves that P_i is honest. If we did not include the condition $P_i \in \text{Pol.Honest}$ and had used the alternative justifier predicate

$$J'(P_i, \text{NoMsg}^{(j)}, \text{Pol}) \equiv \text{Pol} = \overset{j}{\nrightarrow},$$

then the corrupted parties could cook up a polariser Pol with $\text{Pol} = \overset{j}{\nrightarrow}$, i.e., $P_j \in \text{Pol.Corrupt}$, even for an honest P_j . And then P_i would via J' have to accept P_j as corrupted. This was not the predicate we intended. However, by Lemma 1 we have that if $P_j \in \text{Pol.Corrupt}$ *and* P_i is honest then also $P_i \in \text{Pol.Corrupt}$. Therefore P_i can rule out such cooked up Pol by checking that $P_i \in \text{Pol.Honest}$. But for this we need to give J as input the name of the party doing the check. This is the only reason why we give J the name P_i of the checker as input. Note that we require that if any honest party accepts a proof then they would all do it, what we called transferability. Note that Eq. (1) is transferable. Namely, for any two honest parties P_i and P'_i we have that $J(P_i, \text{NoMsg}^{(j)}, \text{Pol}) \equiv J(P'_i, \text{NoMsg}^{(j)}, \text{Pol})$ as it follows from Lemma 1 that $P_i \in \text{Pol.Honest}$ if and only if $P'_i \in \text{Pol.Honest}$. This does *not* hold if P_i is honest and P'_i is corrupted, or *vice versa*, which is why we only require transferability between honest parties. Note that we use $J(m, \pi) = \top$ to mean that $J(P_i, m, \pi) = \top$.

Definition 4 (Justified Inputs/Outputs). We say that a protocol Π has justified inputs if it takes an input justifier J_{In} as parameter and works for any justifier predicate J_{In} . We write $\Pi_{J_{\text{In}}}$ to specify the value of J_{In} being used in a given run. When a protocol $\Pi_{J_{\text{In}}}$ with justified inputs is being called by an honest party P_i with input x_i then x_i must be of the form $x_i = (m_i, \pi_i)$ such that $J_{\text{In}}(P_i, m_i, \pi_i) = \top$. We say that a protocol Π has an output justifier if the protocol, as part of its code, specifies a justifier predicate J_{Out} . We denote the output justifier of Π by $\Pi.J_{\text{Out}}$. We say that a protocol P_i has justified outputs if it has an output justifier and the outputs y_i of honest P_i are of the form $y_i = (m_i, \pi_i)$ and it always holds that $\Pi.J_{\text{Out}}(P_i, m_i, \pi_i) = \top$ after a run of the protocol with a PPT adversary.

An important tool in our protocols is that justified outputs can be passed on to other parties. Therefore, not even adversarial parties should be able to claim wrong outputs. At least they should not be able to convince any honest party about a wrong output. The following notion is later used to phrase this.

Definition 5 (Adversarial Justified Output (AJO)). Let Π be a protocol with an output justifier J_{Out} and let $\mathcal{A}$ be a PPT adversary. Consider the following experiment: Execute

Π with $\mathcal{A}$ in the role of the adversary. When all honest parties P_i have produced an output $y_i = (m_i, \pi_i)$, give all y_i to $\mathcal{A}$. We say that $\mathcal{A}$ generates ℓ adversarial justified outputs (AJOs) $(m^1, \dots, m^\ell)$ if it outputs $(P^1, m^1, \pi^1), \dots, (P^\ell, m^\ell, \pi^\ell)$ such that for all $j = 1, \dots, \ell$ such that P^j is honest and $\Pi.J_{\text{Out}}(P^j, m^j, \pi^j) = \top$. Otherwise, we say that no outputs were generated.

Note that the triple (P^j, m^j, π^j) with $\Pi.J_{\text{Out}}(P^j, m^j, \pi^j) = \top$ does not mean that P^j produced the output (m^j, π^j) . It merely means that P^j would *accept* the output (m^j, π^j) . By transferability of justifier predicates this means that any other honest party would accept too. So in (P^i, m^i, π^i) the value if P_i does not really matter, except that we require that P_i is honest. Note also that if a property holds for all AJOs it also holds for honest outputs as the adversary are given the honest outputs and can reuse them as a triple in $(P^1, m^1, \pi^1), \dots, (P^\ell, m^\ell, \pi^\ell)$.

4 Send Transferable Messages

We now present a protocol which forces a potentially corrupt sender to send a message, solving the *missing message problem* discussed earlier. Throughput, we let NoMsg be a designated symbol where $\text{NoMsg} \notin \{0, 1\}^*$. We use it to signal that a sender was corrupt and did not send a normal message. Ultimately, NoMsg could be mapped to a normal message, like the empty string, but it helps the exposition to assume $\text{NoMsg} \notin \{0, 1\}^*$. We also use another such symbol $\perp$ and assume that $\perp \notin \{0, 1\}^*$ and $\perp \neq \text{NoMsg}$. The protocol allows that a corrupted sender makes honest receivers receive several messages.

Definition 6 (Send Transferable Message Protocol). Let $\Pi_{J_{\text{Msg}}}$ be a protocol run among n parties $P_1, \dots, P_n$ where J_{Msg} is the parametrisable input justifier predicate. Assume that $\Pi_{J_{\text{Msg}}}$ specifies a designated sender P_s holding an input $m \in \{0, 1\}^*$ along with a justifier π such that $J_{\text{Msg}}(P_s, m, \pi) = \top$. Parties P_i may generate several outputs y_i in several rounds. The protocol specifies an output justifier predicate $\Pi.J_{\text{Out}}$.

- **Correctness:** Honest P_i outputs elements of the form $y_i = (m_i, \pi_i)$, where $m_i \in \{0, 1\}^* \cup \{\text{NoMsg}\}$ and $\pi_i \in \{0, 1\}^*$.
- **Justified Output:** Outputs are justified. When honest P_i outputs $y_i = (m_i, \pi_i)$ then $J_{\text{Out}}(P_i, m_i, \pi_i) = \top$.
- **Justified Message:** Only justified inputs can appear in justified outputs. For all AJOs $y_i = (m_i, \pi_i)$ either $m_i = \text{NoMsg}$ or the justifier π_i is of the form $\pi_i = (\pi_i^1, \pi_i^2)$ and $J_{\text{Msg}}(P_i, m_i, \pi_i^1) = \top$ for all honest P_i .
- **Validity:** Honest senders manage to send their intended message and only that message. If P_s is honest and has input (m, π) , then all honest parties P_j have output $y_j = (m_j, \pi_j)$ with $m_j = m$. Furthermore, for all AJOs m' it holds that $m' = m$.
- **Agreement:** All outputs are the same or NoMsg . For all AJOs m^1 and m^2 it holds that $m^1 = \text{NoMsg}$ or $m^2 = \text{NoMsg}$ or $m^1 = m^2$.

We say that Π is a send transferable message with agreement (STMA) protocol with J_{Out} -justified output if it has the above properties. If it lacks agreement we call it an STM protocol.

If an STMA protocol has the additional property that $m \neq \text{NoMsg}$ for all AJOs, then we call it a justifiable broadcast. We call an output a legal STM output if it has the correctness and justified output properties.

Remark 2. Note that justifiable broadcast ensures that all outputs are the same, so it implies the notion of broadcast in Definition 1. It additionally has input and output justifiers, which will be convenient when using it as sub-protocol. It is straight forward to see that we can create a justified broadcast protocol $\text{DSC}_{\mathbf{P}_s, J_{\text{Msg}}}$ by using Dolev-Strong with sender $\mathbf{P}_s$ and $t = n - 1$ corruptions and where parties only accept signatures from $\mathbf{P}_s$ on (m, π_{Msg}) where $J_{\text{Msg}}(m, \pi_{\text{Msg}}) = \top$. The round complexity is $\mathcal{O}(n)$. On messages of length ℓ the communication complexity of DSC is $\mathcal{O}(n^2\ell + n^3\lambda)$, as each party sends the message to each other party at most once and sends a signature chain of size $n\lambda$ to each other party at most once. We use this protocol later.

Remark 3. Note that honest parties have input $m \neq \text{NoMsg}$, so by Validity, if output $m = \text{NoMsg}$ can be justified, then $\mathbf{P}_s$ is corrupt. Furthermore, if m_1 and $m_2 \neq m_1$ can be justified then by Validity $\mathbf{P}_s$ is corrupt.

Remark 4. Generic transformation into *all justified messages delivered*.

Remark 5. We have required that for all AJOs $y_i = (m_i, \pi_i)$ either $m_i = \text{NoMsg}$ or the justifier π_i is of the form $\pi_i = (\pi_i^1, \pi_i^2)$ and $J_{\text{Msg}}(\mathbf{P}_i, m_i, \pi_i^1) = \top$ for all honest $\mathbf{P}_i$. Note that the definition does not restrict what π_i^2 is or how it affects the output of J_{Out} . Typically π_i^2 will be a protocol dependent value proving that the message m_i resulted from running Π and J_{Out} will check that this is the case. Typically π_i^2 also contains a signature on m_i from $\mathbf{P}_s$ to ensure validity.

4.1 Polariser Cast

We present a STM protocol PC called *polariser cast*. The protocol will proceed roughly as follows.

1. The sender signs its justified input and sends it to all parties.
2. If the sender $\mathbf{P}_s$ did not send a justified signed input in round 0 then each $\mathbf{P}_j$ accuses $\mathbf{P}_s$.
3. If $\mathbf{P}_s$ should have been accused but a party $\mathbf{P}_j$ did not accuse $\mathbf{P}_s$ in round 1, then all parties $\mathbf{P}_k$ will accuse $\mathbf{P}_j$ in round 2, unless $\mathbf{P}_j = \mathbf{P}_s$ such that it was already accused, *et cetera*.
4. Since only corrupted parties are accused, at some point there are no more parties to accuse, and at this point an output can be computed. Either $\mathbf{P}_s$ sent a signed message or $\mathbf{P}_s$ can be moved to **Corrupt** along with all parties with enough accusations.
5. This gives each party an output candidate, but different honest parties might hold different signed messages m .
6. The parties exchange their output candidates, and if some $\mathbf{P}_j$ has different signed values in any of them, then this is used as a transferable proof that $\mathbf{P}_s$ is corrupt.

Before describing the protocol in detail we give some helping definitions. During the protocol each P_i will keep a set S of received well-formed *elements*.

Definition 7 (Well-formed elements). We call an element e well-formed if it is of one of the following two forms.

Inputs: $e = (\text{IN}, m, \pi, \sigma)$, where $\text{Ver}_{\text{vk}_s}((\text{IN}, m), \sigma) = \top$ and $J_{\text{Msg}}(P_s, m, \pi) = \top$.

Accusations: $e = (\text{ACC}, P_i, P_j, \sigma_i)$, where $\text{Ver}_{\text{vk}_i}((\text{ACC}, P_i, P_j), \sigma_i) = \top$.

Each P_i has its own version of S . When we need to distinguish it we denote it by S_i . We use S_i^r to denote the value of S_i at P_i in round r . We define some helper functions used in the protocol.

Detect corruption: The function $\text{ToAccuse}^r(S) \subset \mathbb{P}$ takes as input a set of well-formed elements and a round number r and computes a set of parties $\mathcal{P} \in \mathbb{P}$, which we think of as being corrupt.

Complete: We call a set of well-formed elements S *complete* if there are no more parties to accuse, i.e., $\text{Complete}(S) \equiv \exists r > 0 (\text{ToAccuse}^r(S) = \emptyset)$.

Output: The function $\text{Out}(S)$ takes as input a complete set of well-formed elements and computes a possible output of PC, i.e., if $\text{Complete}(S)$ then $\text{Out}(S) = (m, \pi)$ where π is a signature on m under vk_s or $\text{Out}(S) = (\text{NoMsg}, \text{Pol})$ where $\text{Pol} = \overset{s}{\nrightarrow}$.

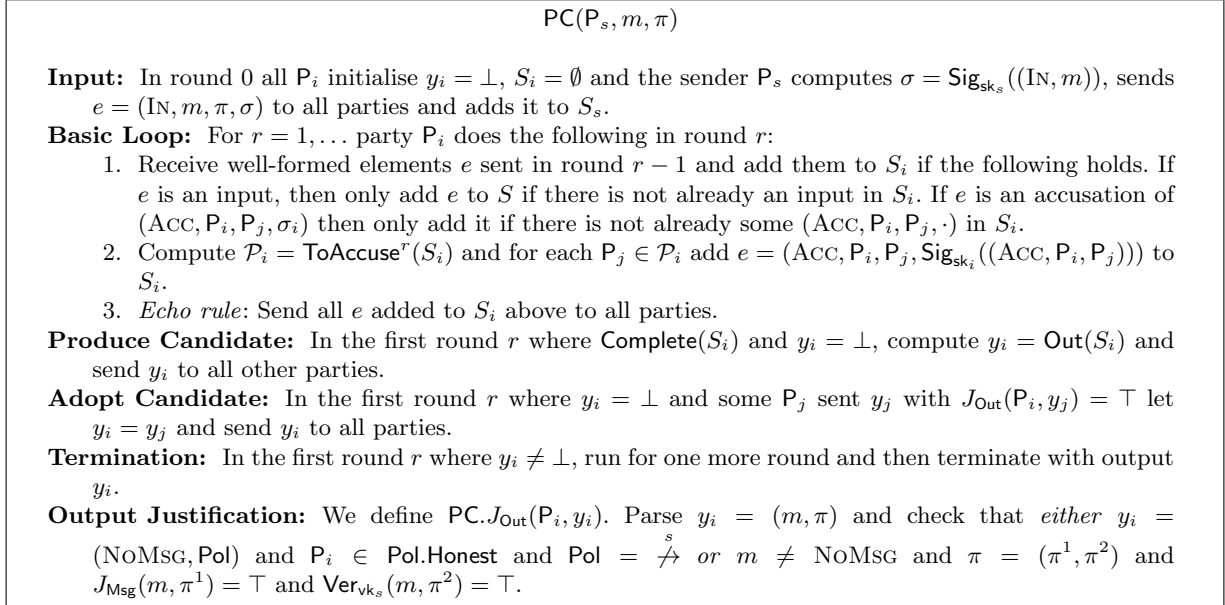


Fig. 1. A Sending Transferable Message Protocol.

The protocol is given in Fig. 1. We now proceed to define **ToAccuse** and **Out**. We use a tree-based definition where **Tree** is a tree of missing accusations. What messages are missing depends on what round we are in, so the function Tree^r also depends on r . The nodes of the tree will be elements $(P, \rho) \in \mathbb{P} \times \mathbb{N}$.

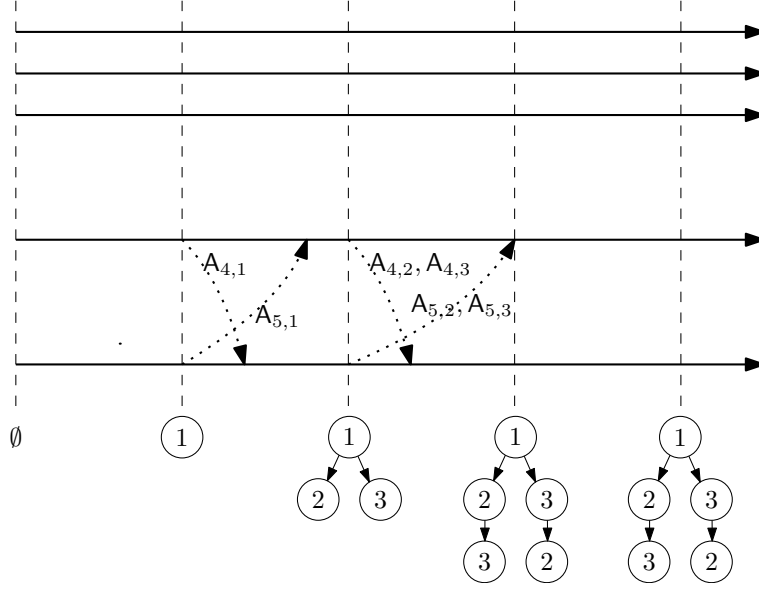


Fig. 2. Party P_1 is the sender. Parties P_1 , P_2 , and P_3 are corrupted. Their timelines are shown as the top three. Parties P_4 and P_5 are honest and their timelines shown at the bottom. Time runs from left to right and vertical dashed lines are round separators with the first one showing the beginning of round 0. To not clutter the figure, we do not show messages sent *to* corrupted parties. Below the timelines we show the tree built by P_5 . In round 0 it is empty. In round 0 party P_1 does not send its signature σ . Therefore P_5 adds the root $(P_1, 0)$. We are then in round 1, so all parties in leafs of paths of length 1 should be accused, i.e., P_1 . In round 1 party P_4 therefore accuses P_1 and P_5 also accuses P_1 . The corrupted parties do not accuse P_1 . Therefore P_5 adds edges from $(P_1, 0)$ to $(P_2, 1)$ and $(P_3, 1)$. We are then in round 2, so all parties in leafs of paths of length 2 should be accused, so both honest parties accuse P_2 and P_3 . The corrupted parties do not accuse. Therefore P_3 's missing accusation of P_2 is added as an edge and P_2 's missing accusation of P_3 is added as an edge. Note that for instance P_1 's missing accusation of P_2 is not added as an edge as we only add parties not already on a path. We are then in round 3 so parties in leafs on paths of length 3 should be accused, i.e., P_3 and P_2 . However, no accusations are actually sent as equivalent accusations were sent already. We are then in round 4. No new nodes are added. Parties in leafs of paths of length 4 should be accused. There are no such paths, so the accusation is considered complete, and the protocol ends. We have $\text{Alive} = \{P_4, P_5\}$, $\text{Corrupt} = \{P_1, P_2, P_3\}$, and $\text{Accuse} = \{A_{4,1}, A_{5,1}, A_{4,2}, A_{4,3}, A_{5,2}, A_{5,3}\}$, so we have a legal P_1 -polariser.

Definition 8 (Tree Function). *The output of $\text{Tree}^r(S)$ is computed as:*

1. Let T be the empty tree.
2. If $\nexists(\text{IN}, m, \cdot, \cdot) \in S$ then add $(P_s, 0)$ to T as the root.
3. For $\rho = 1, \dots, r$:
 - (a) For all $\text{path} \in T$ with $|\text{path}| = \rho$:
 - i. Let $(P_j, \rho - 1) = \text{leaf}(\text{path})$.
 - ii. For all $P_k \in \mathbb{P}$ where $(P_k, \cdot) \notin \text{path}$ and $\nexists(\text{ACC}, P_k, P_j, \cdot) \in S$, add the edge $((P_j, \rho - 1), (P_k, \rho))$ to T .
4. Output T .

We think of $\text{Tree}^r(S)$ as the tree of missing elements relative to an honest run of $\text{PC}(P_s, \dots)$. As an example, if P_s is honest it should send $(\text{IN}, m, \cdot, \cdot)$ in round 0, so if $\nexists(\text{IN}, m, \cdot, \cdot) \in S$ then we add $(P_s, 0)$ to signify that P_s omitted a message in round 0. Therefore, if the tree has the path $((P_s, 0))$ then all P_k should have accused P_s in round 1. If P_k does not do this, then in the iteration of the loop with $\rho = 1$ the path $\text{path} = ((P_s, 0))$ of length 1 will get considered and so will $(P_s, 0) = \text{leaf}(\text{path})$. So if $P_k \neq P_s$ did not accuse P_s then we will have $\nexists(\text{ACC}, P_k, P_s, \cdot) \in S$ and hence $((P_s, 0), (P_k, 1))$ gets added to the tree T . So we add an edge to $(P_k, 1)$ to signify that in round 1 party P_k failed an obligation and then points from $(P_s, 0)$ to say that the obligation was to accuse P_s because P_s failed an obligation in the previous round. The reason that Tree^r depends on r is that some accusation might be missing simply because the parties did not have a chance to send them yet.

We now define $\text{ToAccuse}^r(S)$. To motivate the definition, consider a tree $\text{Tree}^{r-1}(S_i)$ at P_i at the beginning of round r , i.e., after receiving the accusations $(\text{ACC}, \cdot, \cdot)$ sent out in round $r - 1$. If $\text{path} = (\dots, (P_j, r - 1)) \in \text{Tree}^{r-1}(S_i)$ this is because P_j missed an obligation in round $r - 1$. Therefore P_i must accuse P_j . This motivate the following definition

$$\text{ToAccuse}^r(S) = \{ P_j \mid \exists(\dots, (P_j, r - 1)) \in \text{Tree}^{r-1}(S) \} .$$

If a set S is complete then it allows to compute an output as follows.

Definition 9 (Output). *The function $\text{Out}(S)$ is defined as follows.*

1. The input is a complete set S , so we can find the smallest r such that $\text{ToAccuse}^r(S) = \emptyset$.
2. If $r = 1$ then pick $(\text{IN}, m, \sigma, \pi) \in S$ and output $(m, (\sigma, \pi))$.
3. If $r > 1$ then output $(\text{NoMsg}, \text{Pol} = (\text{Alive}, \text{Corrupt}, \text{Accuse}))$, where

$$\begin{aligned} \text{Corrupt} &= \{ P_j \mid \exists(P_j, \cdot) \in \text{nodes}(\text{Tree}^r(S)) \} , \\ \text{Accuse} &= S , \\ \text{Alive} &= \mathbb{P} \setminus \text{Corrupt} . \end{aligned}$$

We proceed to prove PC secure when using the above definitions of ToAccuse , Complete , and Out . Before the proof it may be instructive to consider the example runs of the protocol in Figs. 2 to 4.

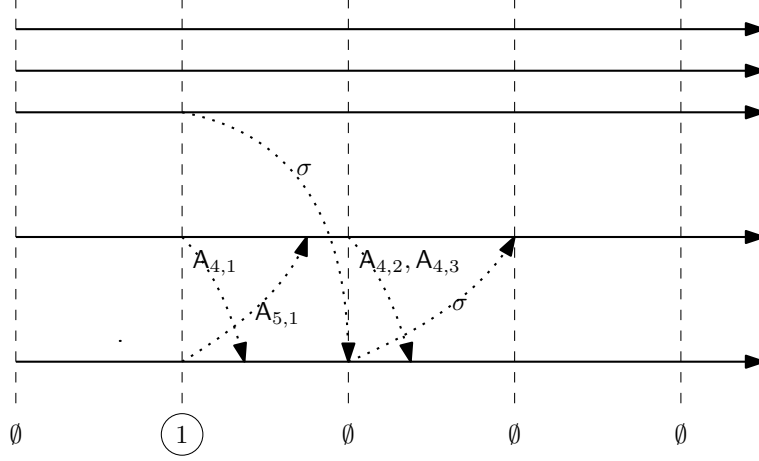


Fig. 3. For notation see Fig. 2. In round 0 party P_1 does not send its signature σ . Therefore P_5 adds the root $(P_1, 0)$. We are then in round 1 and parties P_4 and P_5 accuse P_1 . The corrupted parties do not accuse P_1 , but P_3 forwards a signature σ by P_1 to P_5 . Therefore the tree computed by P_5 in round 2 is again empty. Therefore the accusation is over for P_5 . It outputs σ (it terminates one round later, not shown). Note that P_4 is in the same situation as in Fig. 2. Its tree will look like that of P_5 in round 2 in Fig. 2. So, it accuses P_2 and P_3 . By the echo rule P_5 forwards σ to P_4 which will then have an empty tree by round 3 and outputs σ .

Definition 10 (equivalent sets). Let S and T be two sets of well-formed elements. We say that $S \sqsubseteq T$ if $\exists(\text{IN}, m, \pi, \sigma_s) \in S$ implies that $\exists(\text{IN}, m', \pi', \sigma'_s) \in T$ and $\exists(\text{ACC}, P_i, P_j, \sigma_i) \in S$ implies that $\exists(\text{ACC}, P_i, P_j, \sigma'_i) \in T$. We call two sets equivalent if $S \sqsubseteq T$ and $T \sqsubseteq S$. We call two elements equivalent if $\{e_1\}$ and $\{e_2\}$ are equivalent.

Lemma 2 (propagation lemma). For all honest P_i and P_j and rounds $r > 0$ reached in the protocol it holds that $S_i^{r-1} \sqsubseteq S_j^r$.

Proof. This follows from the fact that all elements s added to S_i get forwarded to P_j and if s is considered well-formed by P_i then it is also considered well-formed by P_j . Therefore s is added to S_j^r , unless S_j^r contains an equivalent element. $\square$

Lemma 3 (tree monotonicity lemma). For all sets of well-formed elements S, T and all $r \geq 0$ it holds that

$$S \sqsubseteq T \implies \text{Tree}^r(T) \subseteq \text{Tree}^r(S) .$$

Proof. Note that for an object o , root or edge, to be included in $\text{Tree}^r(T)$ it is required that some element is *missing* in T , i.e., $\nexists(\text{IN}, m, \cdot, \cdot) \in T$ or $\nexists(\text{ACC}, P_k, P_j, \cdot) \in T$. Since $S \sqsubseteq T$ these conditions imply that $\nexists(\text{IN}, m, \cdot, \cdot) \in S$ and $\nexists(\text{ACC}, P_k, P_j, \cdot) \in S$, so the same object o gets included in $\text{Tree}^r(S)$. $\square$

The following corollary is important in showing that honest do not accuse honest parties. The tree $\text{Tree}^{r-1}(S_i^{r-1})$ is the tree that P_i used in round $r - 1$ to calculate who it should accuse. The tree $\text{Tree}^{r-1}(S_j^r)$ is the tree that S_j uses in round r to calculate who S_i ought to have sent an accusation against—it uses the same function Tree^{r-1} , but its own set S_j^r . If

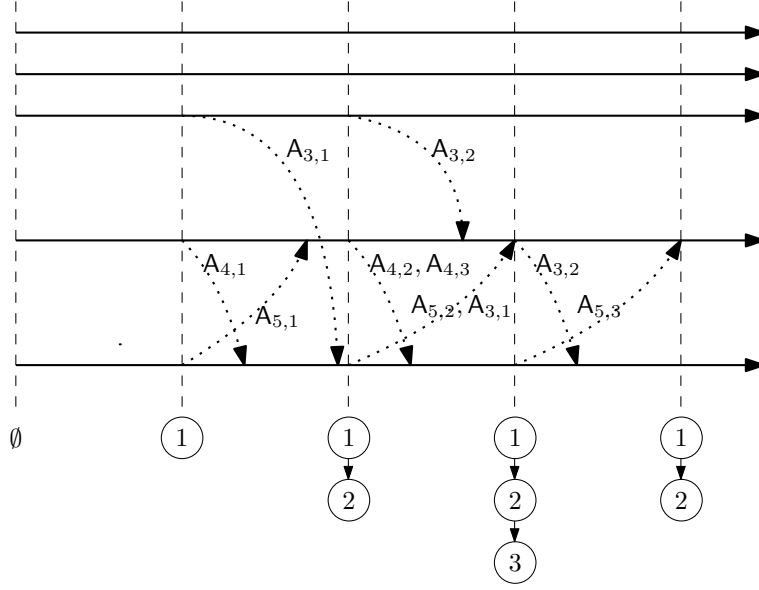


Fig. 4. For notation see Fig. 2. In round 0 party P_1 does not send its signature σ . Therefore P_5 adds the root $(P_1, 0)$. We are then in round 1 and parties P_4 and P_5 accuse P_1 . Party P_3 accuses P_1 but sends the accusation only to P_5 . Party P_2 does not accuse P_1 . Party P_5 adds an edge representing the missing accusation of P_1 by P_2 . Party P_4 being in the same situation as in Fig. 2 accuses P_2 and P_3 . Party P_5 accuses all parties which are leaves on paths of length 2, i.e., party P_2 . It also forwards $A_{3,1}$ because of the echo rule. Now P_3 accuses P_2 but only towards P_4 . Therefore P_5 is missing the accusation of P_2 by P_3 and adds an edge to represent it. It then has a path of length 3 in round 3 and thus accuses P_3 . But in the same round P_4 forwards $A_{3,2}$ because of the echo rule. Therefore by round 4 the tree computed by party P_5 is back to height 1 and the protocol ends for P_5 . It outputs $\text{Alive} = \{P_3, P_4, P_5\}$, $\text{Corrupt} = \{P_1, P_2\}$ and $\text{Accuse} = \{A_{4,1}, A_{5,1}, A_{3,1}, A_{5,2}, A_{4,2}, A_{5,3}, A_{3,2}\}$, which is a legal P_1 -polariser.

$\text{Tree}^{r-1}(S_j^r) \subseteq \text{Tree}^{r-1}(S_i^{r-1})$ then P_j does not expect to receive any accusations which are not sent.

Corollary 1. *For all honest P_i and P_j and rounds $r > 0$ reached in the protocol it holds that*

$$\text{Tree}^{r-1}(S_j^r) \subseteq \text{Tree}^{r-1}(S_i^{r-1}) .$$

Proof. By the preceding lemmas we have that $S_i^{r-1} \subseteq S_j^r$ and that $S \subseteq T \implies \text{Tree}^\rho(T) \subseteq \text{Tree}^\rho(S)$ for all S, T and ρ . Set $S = S_i^{r-1}$, $T = S_j^r$ and $\rho = r - 1$. $\square$

Lemma 4. *If P_i is honest and accuses P_j then P_j is corrupted.*

Proof. By construction, if P_i accuses P_j in round r then $P_j \in \text{ToAccuse}^r(S_i^r)$. By definition this means that $\exists(\dots, (P_j, r-1)) \in \text{Tree}^{r-1}(S_i^r)$. If $r = 1$ then this implies that $\exists((P_j, 0)) \in \text{Tree}^0(S_i^r)$, and hence $P_j = P_s$, as only P_s can occur in the root. Therefore $\nexists(\text{IN}, m, \cdot, \cdot) \in S_i$. Hence, $P_j = P_s$ did not send its signed input to P_i in round 0. Therefore $P_j = P_s$ is corrupted. If $r > 1$ then we have that $\exists(\dots, (P_k, r-2), (P_j, r-1)) \in \text{Tree}^{r-1}(S_i^r)$. By construction (see Item 3(a)ii in Definition 8) this implies that $\exists(\dots, (P_k, r-2)) \in \text{Tree}^{r-1}(S_i^r)$ and $\nexists(\text{ACC}, P_j, P_k, \cdot) \in S_i^r$. If P_j is honest this implies that $\exists(\dots, (P_k, r-2)) \in \text{Tree}^{r-1}(S_j^{r-1})$, as $\text{Tree}^{r-1}(S_i^r) \subseteq \text{Tree}^{r-1}(S_j^{r-1})$. Therefore $P_k \in \text{ToAccuse}^{r-1}(S_j^{r-1})$. So if P_j is honest it sent $(\text{ACC}, P_j, P_k, \cdot)$ to P_i in round $r-1$. This contradicts $\nexists(\text{ACC}, P_j, P_k, \cdot) \in S_i^r$. $\square$

The following lemma shows that when the set S is complete at an honest party such that it terminates, then $\text{Out}(S)$ produces a correct output, i.e., if there is no signature in S , then a polariser is produced proving that P_s is corrupt.

Lemma 5 (justified output). *If S is held by an honest party P_i and $\text{Complete}(S)$, such that P_i produces output $\text{Out}(S)$, then $\text{PC.J}_{\text{Out}}(P_i, \text{Out}(S))$.*

Proof. We want to prove that $\text{PC.J}_{\text{Out}}(P_i, \text{Out}(S)) = \top$. This means that if we let $y_i = \text{Out}(S)$, as defined in Definition 9, then we have to make sure that $\text{PC.J}_{\text{Out}}(P_i, y_i)$ where PC.J_{Out} is defined in **Output Justification** in Fig. 1. We write this out. First parse $y_i = (m, \pi)$. We then have to prove that *either* 1) $y_i = (\text{NoMsg}, \text{Pol})$ and $P_i \in \text{Pol.Honest}$ and $\text{Pol} = \overset{s}{\nrightarrow}$ or 2) $m \neq \text{NoMsg}$ and $\pi = (\pi^1, \pi^2)$ and $J_{\text{Msg}}(m, \pi^1) = \top$ and $\text{Ver}_{\text{vk}_s}(m, \pi^2) = \top$.

By $\text{Complete}(S)$ there exists r such that $\text{ToAccuse}^r(S) = \emptyset$. Assume that $r = 1$. From $\text{Complete}(S)$ we get that $\text{ToAccuse}^1(S) = \emptyset$. This implies that $\nexists(\dots, (P_j, r-1)) \in \text{Tree}^{r-1}(S)$ which for $r = 1$ means that $\nexists((P_j, 0)) \in \text{Tree}^0(S)$, which by the construction of the tree T in the algorithm Tree^0 defined in Definition 8 means that it is *not* the case that $\nexists(\text{IN}, m, \cdot, \cdot) \in S$. So, there is some well-formed $(\text{IN}, m, \cdot, \cdot) \in S$. Therefore the output is of the form in case 2 above.

Assume then that $r > 1$. From $\text{Complete}(S)$ we get that $\text{ToAccuse}^r(S) = \emptyset$ and by r being minimal we have that $\text{ToAccuse}^{r-1}(S) \neq \emptyset$. From $\text{ToAccuse}^{r-1}(S) \neq \emptyset$, it follows that there is at least one path in $\text{Tree}^{r-1}(S)$ and hence also a root. Therefore $(P_s, 0) \in \text{nodes}(\text{Tree}^{r-1}(S))$ and hence $P_s \in \text{Pol.Corrpt}$. We therefore just have to show that Pol is a

legal polariser. Completeness follows from $\text{Alive} = \mathbb{P} \setminus \text{Corrupt}$. Since S contains only well-formed elements and $\text{Accuse} = S$ by Definition 9, for justifiability it is sufficient to prove that for all $(P_i, P_j) \in \text{Alive} \times \text{Corrupt}$ it holds that $(\text{Accuse}, P_i, P_j, \cdot) \in S$. So, assume that $(P_i, P_j) \in \text{Alive} \times \text{Corrupt}$. This implies that $(P_j, \rho) \in \text{nodes}(\text{Tree}^\rho(S))$ for some $\rho < r$. We argue that this implies that S contains $(\text{Acc}, P_i, P_j, \cdot)$. Assume to the contrary that S does not contain $(\text{Acc}, P_i, P_j, \cdot)$. Then it would be the case that $((P_j, \rho), (P_i, \rho + 1)) \in \text{Tree}^r(S)$ by Item 3(a)ii in Definition 8 unless $(P_i, \cdot)$ was already on the path in question (but $(P_i, \cdot)$ being on the path contradicts $P_i \in \text{Alive}$ as we added to Corrupt all parties on all paths by construction of Definition 9). But if $((P_j, \rho), (P_i, \rho + 1)) \in \text{Tree}^r(S)$ then from $\rho + 1 \leq r$ and because we assume that $(\text{Acc}, P_i, P_j, \cdot) \notin S$ it is not the case that $\text{ToAccuse}^r(S) \neq \emptyset$, as we would have $P_i \in \text{ToAccuse}^r(S)$ by construction. Since we have as premise that $\text{ToAccuse}^r(S) = \emptyset$ it follows that $(\text{Acc}, P_i, P_j, \cdot) \in S$, as desired. $\square$

Theorem 1. *The protocol PC is an STM protocol for $t < n$. Furthermore, assume that PC has inputs (m, π) with $|(m, \pi)| \leq \ell$. Let λ be the length of a signature. Then the protocol has communication complexity $\mathcal{O}(n^2\ell + n^4\lambda)$ and the size of the justified output is at most $\mathcal{O}(\ell + n^2\lambda)$. Not counting the accusation of a polariser the justified output is at most $\mathcal{O}(\ell)$. The protocol uses at most $f + 2$ rounds.*

Proof. Correctness follows by construction of Out . Justified output follows from Lemma 5. Justified message follows by construction of J_{Out} . Validity follows by the fact that if P_s is honest then in any polariser Pol accepted by an honest party it holds that $P_s \in \text{Alive}$ by Lemma 4. We then count communication complexity. We ignore constant factors in the counting. In round 0 party P_s sends to all parties its input of length ℓ and a signature. This is the sending of $n(\ell + \lambda)$ bits. During the basic loop each party forwards at most one well-formed input from P_s to other parties, so this is at most $n^2(\ell + \lambda)$ bits. Besides this, each P_i might send an accusation of each P_j which will then be relayed by each other party. This is the flooding of at most $n^4\lambda$ bits. The output consists of at most one well-formed input of P_s so is at most $\ell + n^2\lambda$ bits. In Produce Candidate and Adopt Candidate each party sends it to at most n other parties, yielding communication at most $n^2\ell + n^4\lambda$. We consider round complexity. For illustration consider the easy case for $f = 0$. In the first round P_s sends its signed input to all parties and in round 2 all parties send their adopted candidate y_i . This is $2 = f + 2$ rounds. Note then that if the protocol in rounds $r \geq 1$ do not send an adopted candidate it is because $\neg \text{Complete}(S)$, which implies that $\text{ToAccuse}^r(S) \neq \emptyset$ which by Item 3(a)ii in Definition 8 implies that P_k for which $(P_k, \cdot) \notin \text{path}$ is added to T in Tree^r , extending path by length 1. After this path contains one more corrupted party. There are at most f corrupted parties. So this adds at most f extra rounds, for a total of at most $2 + f$. $\square$

The bulk for the communication of PC is the flooding of up to n^2 accusations. It turns out this can be compressed accross multiple runs of PC as each accusation needs only be sent once.

Lemma 6 (Amortized Communication Complexity). *Assume that in the life time of the system ι instances $\text{PC}^1, \dots, \text{PC}^\iota$ are run and that PC^i has inputs (m^i, π^i) with $|(m^i, \hat{\pi}^i)| \leq$*

ℓ_i , where $\hat{\pi}^i$ is π^i with all accusations removed. Then the communication complexity of running all ι copies can be compressed to $\mathcal{O}(n^2 \sum_i \ell_i + n^4 \lambda)$ without affecting the security of the protocol.

Proof. If in a given system an accusation $e = (\text{ACC}, P_i, P_j, \sigma_i)$ was sent as part of running one PC, then do not send it again. Also, add it to all sets S_i in all copies. Also, do not send it as part of the justifications after it was sent once. If $e = (\text{ACC}, P_i, P_j, \sigma_i)$ was received once then add it to all incoming justifications. This can be seen to not affect the execution of the protocol. If π justified m then it also does so after adding one more accusation. This way, overall, each of the n^2 possible $e = (\text{ACC}, P_i, P_j, \sigma_i)$ will be sent at most once per pair of parties for a total of $n^4 \lambda$ communication. Then notice that extra to accusations the protocol only send something when an input e is added to S . This happens at most n times as we do not add e if S already has an input element. And when it happens (m^i, π^i) is sent to at most n parties. Sending one (m^i, π^i) counts as ℓ_i bits of extra communication as we already accounted for the accusation. $\square$

Remark 6 (Further Amortisation). We will later discuss how communication can be amortised more than the above lemma reflects. This is because justifiers often consist of values which have already been sent. If a justifier for an outgoing message contains justifiers for earlier incoming messages and the corresponding justifiers were already resent earlier, then there is no reason to send them again. Thereby each “basic” justifier will be sent at most n^2 times. These optimisation are better done in context of concrete protocols, so we add the details later.

5 Send Transferable Messages with Agreement

We now show how to add agreement to any STM protocol by giving an STMA protocol using STM as sub-protocol. The protocol is given in Fig. 5.

Theorem 2. *Protocol $\text{STMA}_{J_{\text{Msg}}, P_s}$ is an STMA protocol. Assuming that STMA has inputs (m, π) with $|m, \pi| \leq \ell$, it has communication complexity $\mathcal{O}(n^3 \ell + n^4 \lambda)$ and the size of the justified output is at most $\mathcal{O}(n \ell + n^2 \lambda)$. If P_s is honest, it uses 2 rounds.*

We have to argue Correctness, Justified Output, Justified Message, Validity, Agreement and the claimed communication. Correctness follows by construction. Justified Output too: the constructed justifier δ clearly matched the defined predicate $\text{STMA}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$. Justified Message is also by construction. When $m \neq \text{NoMsg}$ then by construction $\delta = (\pi, \cdot)$, where $J_{\text{Msg}}(m, \pi) = \top$. We now argue Validity and Agreement.

Lemma 7 (Validity). *Protocol $\text{STMA}_{J_{\text{Msg}}, P_s}$ in Fig. 5 is valid.*

Proof. By the properties of STM, any AJO of $\text{STM}_{J_{\text{Msg}}, P_s}$ is justified by the justifier predicate $\text{STM}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$ defined by the protocol. By definition we have to show that if P_s is honest and tries to send m then all AJOs for $\text{STMA}_{J_{\text{Msg}}, P_s}(m, \pi_{\text{Msg}}) \cdot J_{\text{Out}}$ have message m . We show this in three steps. In step 1 we show that if P_s is honest then all AJOs for $\text{STM}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$

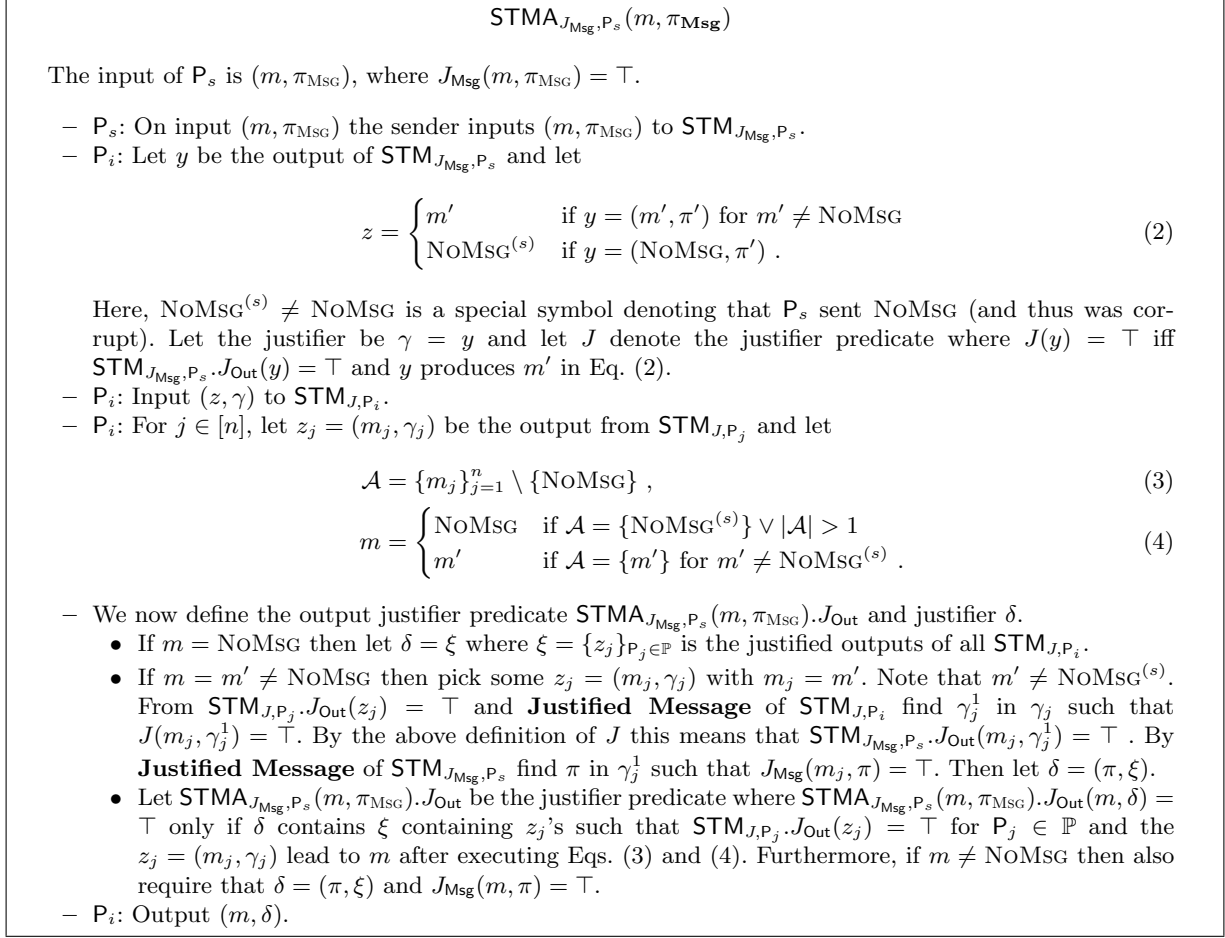


Fig. 5. An STMA protocol.

are for m . In step 2 we argue that this means that all AJOs for $\text{STM}_{J, P_i} \cdot J_{\text{Out}}$ are for m or NoMsg and that it is for m when P_i is honest. In step 3 we argue that this means that if $\text{STMA}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$ has an AJO for m'' then $m'' = m$.

Arguing validity also includes arguing that all honest parties get output. This follows by construction and the fact that all sub-protocols terminate. We do not go further into this below.

Step 1. Suppose P_s is honest and has input (m, π_{Msg}) . In this case, P_s inputs (m, π_{Msg}) to $\text{STM}_{J_{\text{Msg}}, P_s}$ such that $J_{\text{Msg}}(m, \pi_{\text{Msg}}) = \top$. By validity of $\text{STM}_{J_{\text{Msg}}, P_s}$, every AJO for $\text{STM}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$ is therefore of the form $y = (m, \cdot)$.

Step 2. By the above argument all honest parties P_i input (m, γ) to STM_{J, P_i} . By validity of STM_{J, P_i} , every AJO (m, γ_i) for $\text{STM}_{J, P_i} \cdot J_{\text{Out}}$ will have $J(m, \gamma_i) = \top$ when P_i is honest. Furthermore, even for corrupt P_j it follows from **Justified Message** that an AJO for $\text{STM}_{J, P_j} \cdot J_{\text{Out}}$ is either for NoMsg or has $J(m', \gamma_j) = \top$. If $J(m', \gamma_j) = \top$ then by definition $\text{STM}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}(m', \gamma_j) = \top$. Therefore, as argued in step 1, $m' = m$. All in all, this gives us that all AJOs m' for $\text{STM}_{J, P_i} \cdot J_{\text{Out}}$ are for $m' = m$ or $m' = \text{NoMsg}$ and that $m' = m$ when P_i is honest.

Step 3. Assume then an AJO (m'', δ) for $\text{STMA}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$. By definition of $\text{STMA}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$ this means that δ contains $\xi = \{z_j\}_{P_j \in \mathbb{P}}$ and that $\text{STM}_{J, P_j} \cdot J_{\text{Out}}(z_j) = \top$ for $P_j \in \mathbb{P}$ and the $z_j = (m_j, \gamma_j)$ in ξ leads to m'' after executing Eqs. (3) and (4). There is at least one honest party P_i , which by step 2 implies that $m_i = m \neq \text{NoMsg}$. Again by step 2, all m_j are either NoMsg or m . This means that after executing Eq. (3) we have $\mathcal{A} = \{m\}$. As argued above, after executing Eqs. (3) and (4) we get m'' . By construction of Eq. (4) we conclude that $m'' = m$. $\square$

Lemma 8 (Agreement). *Protocol $\text{STMA}_{J_{\text{Msg}}, P_s}$ in Fig. 5 has agreement.*

Proof. We have to show that if $(m^1, \cdot)$ and $(m^2, \cdot)$ are AJOs for $\text{STM}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$ then $m^1 = \text{NoMsg}$ or $m^2 = \text{NoMsg}$ or $m^1 = m^2$. For $b = 1, 2$ assume then an AJO (m^b, δ^b) for $\text{STMA}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$. By definition of $\text{STMA}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}$ this means that δ^b contains $\xi^b = \{z_j^b\}_{P_j \in \mathbb{P}}$ and that $\text{STM}_{J, P_j} \cdot J_{\text{Out}}(z_j^b) = \top$ for $P_j \in \mathbb{P}$ and the $z_j^b = (m_j^b, \gamma_j^b)$ in ξ^b leads to m^b after executing Eqs. (3) and (4). Let $\mathcal{A}^b$ be the value after executing Eq. (3) on ξ^b . There is at least one honest party P_i , which by **Validity** of STM_{J, P_i} gives us that $m_i^1 = m_i^2 \neq \text{NoMsg}$. Namely, when P_i is honest then STM_{J, P_i} can only output the m_i that P_i intends to send, and there is exactly one such m_i . And this honest input value m_i cannot be NoMsg . Let $m_i \neq \text{NoMsg}$ be the common value such that $m_i^b = m_i$. If $m_i = \text{NoMsg}^{(s)}$ then by construction of Eqs. (3) and (4) we get $m^1 = m^2 = \text{NoMsg}$ as $\text{NoMsg}^{(s)} \in \mathcal{A}^b$, so $\mathcal{A}^b = \{\text{NoMsg}^{(s)}\}$ or $|\mathcal{A}^b| > 1$. If $m_i \neq \text{NoMsg}^{(s)}$ then because $m_i \in \mathcal{A}^b$ and by definition of Eq. (4), either $m^b = m_i$ or $m^b = \text{NoMsg}$. In either case we showed what we had to. $\square$

Lemma 9 (Amortised Communication). *Let $\ell = |(m, \pi_{\text{Msg}})|$. Not counting accusations the communication of the protocol is $\mathcal{O}(n^3\ell)$ and the size of a justifier δ is $\mathcal{O}(n\ell)$. Counting also accusations the communication of the protocol is $\mathcal{O}(n^3\ell + n^4\lambda)$ and the size of the justifier δ is $\mathcal{O}(n\ell + n^2\lambda)$.*

Proof. The communication is dominated by that of running the n instances of STM_{J, P_i} . For these the input is (z, γ) . Not counting accusations of a possible justifier the size of (z, γ) is basically that of (m, π_{Msg}) , i.e., $\mathcal{O}(\ell)$. Therefore the communication of one instance is $\mathcal{O}(n^2\ell)$ and the overall communication is $\mathcal{O}(n^3\ell)$. This is again not counting the sending of accusations. Not counting accusations the size of one justified output z_j is $\mathcal{O}(\ell)$. Therefore the size of one justifier δ is $\mathcal{O}(n\ell)$. Using the optimisation of Lemma 6, each of the possible n^2 accusations is each sent at most n^2 times, which adds at most $\mathcal{O}(n^4\lambda)$ to the communication. There is at most n^2 accusations and we only need to add each of them once to the justifier δ , which adds at most $\mathcal{O}(n^2\lambda)$ to the size of the justifier. $\square$

6 Compressed Transferable Justifiers

In this section we introduce an abstraction called *Compressed Transferable Justifiers* which helps to control the communication complexity. In the following sections, we present protocols which run for up to n rounds. In these protocols, justifiers for later rounds consist of justifiers for outputs in previous rounds. Without further care, this could lead to an exponential

blowup in communication complexity. To prevent this from happening, our key technique is to forward all outputs of all STMA subprotocols to all parties and then simply let the justifiers point to those STMA outputs that they are using. This will be based on the three following rules.

STMA Justification: In all protocols which follow, all messages can ultimately be justified by previous outputs of some STMA instance(s). This is based on the observation that each of our protocol subroutines internally only calls STMA for communication. Hence, even though message m might be justified by the output y of some previous sub-protocol different from STMA, y (and therefore m) will then in turn recursively be justified by previous outputs of some STMA instances.

STMA Echo: Whenever a party locally receives an output NoMsg from an STMA instance I , it forwards this message to all other parties. It will also echo any justified NoMsg value for I forwarded to it from other parties. In either case, it only forwards NoMsg if it did not previously forward NoMsg for STMA instance I . Similarly, whenever a party receives a justified output $m \neq \text{NoMsg}$ for an STMA instance I or receives one by forward, it sends m to all other parties if it did not previously send some justified m for STMA instance I . Since by **Agreement** of STMA, each STMA instance has at most one $m \neq \text{NoMsg}$ such that an AJO for m can be produced, it follows that for each STMA instance I , each party forwards to each other party at most two justified outputs (one for NoMsg and one for some m). Furthermore, for all STMA instances, all parties will see the same outputs with at most one round of delay as a party forwards any received message in the round where it sees it the first time.

STMA Justifier Predicates: Finally, we will by design only use so-called *STMA justifiers*, i.e., justifier predicates whose satisfaction depends only on the set of justified outputs y from STMA, but not the particular justifiers π which justify those y . This ensures that it suffices for parties to eventually hold the same set of justified y . They need not end up holding the same justifiers π . This observation is crucial as a given justified y can have exponentially many different justifiers.

Compressed Justifiers: The above rules allow to compress justifiers as follows. If a justifier π for y holds as part of it a justified output (y', π') for a previous STMA instance I' , then we can replace (y', π') , and in particular the justifier π' , by a pointer to instance I' . We simply send (I', b) where $b = 0$ indicates to use NoMsg as the output of I' and $b = 1$ indicates to use a non-NoMsg value as the output of I' . In either case, the **STMA Echo** rule ensures that a receiver of a forwarded compressed justifier will have the referenced STMA outputs available when it receives that justifier. Moreover, the **STMA Justification** and the **STMA Justifier Predicates** rules together ensure that any compressed justifier can be expressed as a combination of STMA outputs and that the evaluation of that justifier will evaluate to the same outcome in the view of any honest party: even if it uses different justifiers it will have the justified y 's needed to satisfy the predicate of the compressed justifier.

We now add more details to the above mechanism. We start with the STMA Echo and Compressed Justifiers. All parties will run the following rules in the background.

Init P_i : In round 0 let S_i be the empty set.

Output P_i : If an instance I of STMA outputs (y, π) and there is not already an element of the form $(I, y, \cdot) \in S_i$, then add (I, y, π) to S_i and forward (I, y, π) to all parties. Note that in this case $I.J_{\text{Out}}(y, \pi) = \top$ by **Justified Output**.

Echo P_i : On receiving (I, y, π) from any party where $I.J_{\text{Out}}(y, \pi) = \top$ and there is not already an element of the form $(I, y, \cdot) \in S_i$, add (I, y, π) to S_i and forward (I, y, π) to all parties.

Compress P_i : When sending a justifier π for instance I take any component of the form (I', y', π') in π , where $(I', y', \pi') \in S_i$, and replace it by (I', b) , where $b = 0$ if $y' = \text{NoMsg}$ and $b = 1$ if $y' \neq \text{NoMsg}$. We write $\hat{\pi} = \text{enc}(\pi, S_i)$ to denote the compressed version of justifier π .

Decompress P_i : When receiving a compressed justifier $\hat{\pi}$ for instance I take any component of the form (I', b) in $\hat{\pi}$ and replace it by some $(I', y', \pi') \in S_i$, where $y' = \text{NoMsg}$ if $b = 0$ and $y' \neq \text{NoMsg}$ if $b = 1$. If no such element exists in S_i then drop the incoming $\hat{\pi}$. We write $\pi' = \text{dec}(\hat{\pi}, S_i)$ and let $\pi' = \perp$ denote that the message was dropped. For notational convenience we assume below that $J(y, \perp) = \perp$ for all justifier predicates.

We call the above the *Compressed Echo System*. In the following, we denote as S_i^r the value of S_i at P_i in round r . We drop r from the superscript if a statement is true for all rounds of r . By design the following holds.

Lemma 10 (Output Propagation). *For all honest parties P_i and P_j , all rounds r , and all STMA instances I it holds that*

$$\exists(I, y, \pi) \in S_i^r \implies \exists(I, y, \pi') \in S_j^{r+1}.$$

Note that compression and subsequent decompression will always reproduce the same y' , by **Agreement**. However, the value of π' might change as the receiver might have another justifier. To argue about this the following notions are helpful.

Definition 11. *Let S and S' be sets of justified STMA outputs. We say that $S \sqsubset S'$ if for all $(I, y, \pi) \in S$ there exists at least one $(I, y, \pi') \in S'$.*

Lemma 11 (Output Correctness). *Let P_i and P_j be honest parties. If $S_i \sqsubset S_j$ and $\hat{\pi} = \text{enc}(\pi, S_i)$ and $\tilde{\pi} = \text{dec}(\hat{\pi}, S_j)$ then $\tilde{\pi}$ is equal to π except that each value of the form (I, y', π') with $I.J_{\text{Out}}(y', \pi') = \top$ in π might have been replaced by some (I, y', π'') with $I.J_{\text{Out}}(y', \pi'') = \top$.*

The following captures that a justifier predicate does not change from true to false if some π' is replaced by π'' as above.

Definition 12 (STMA Justifier Predicate). *We say that J is an STMA justifier predicate if it holds for all $S \sqsubset S'$ and (y, π) that $J(y, \pi) = \top \implies J(y, \text{dec}(\text{enc}(\pi, S), S')) = \top$.*

Combining the above, we easily get the following corollary, which shows correctness of the Compressed Echo System in the sense that whenever a justified (y, π) is sent for an instance I , then a justified (y, π') is received for I .

Corollary 2 (Justification Propagation). *For all STMA justifiers J , honest parties P_i and P_j , all rounds r , and all STMA instances I it holds that*

$$J(y, \pi) = \top \implies J(y, \text{dec}(\text{enc}(\pi, S_i^r), S_j^{r+1})) = \top .$$

Below we assume that instance names I can be represented by $\mathcal{O}(\lambda)$ bits.

Lemma 12 (Communication). *Consider an execution with STMA instances $I_1, \dots, I_\ell$, where ℓ_i is an upper bound on the bit length of an output y_i of instance I_i and prulen_i is an upper bound on the bit-length of a compressed π_i for I_i after also removing all accusations. Then the communication of the above Compressed Echo System is at most*

$$\mathcal{O} \left(n^2 \left(\sum_{i=1}^{\ell} (\ell_i + \text{prulen}_i) \right) + n^4 \lambda \right) .$$

Proof. When applying Lemma 6, each accusation is sent from each party to each party at most once, giving the $n^4 \lambda$ term. Besides this, each different output y of each instance I is sent by each party to each party at most once, and each instance, by **Agreement**, has at most two different justified outputs. This gives at most $\mathcal{O}(n^2 \sum_{i=1}^{\ell} \ell_i)$. Finally, the parts of justifiers π which are not accusations or previous outputs of STMA instances will be sent by each party to each other party at most once, contributing $\mathcal{O}(n^2 \sum_{i=1}^{\ell} \text{prulen}_i)$. $\square$

We finally define a notion of generic justifier. This is just an STMA justifier for an input to a sub-protocol which recursively proves that this input could have been obtained by running the protocol thus far on justified outputs from even earlier sub-protocols. We formalise this in the following.

In all protocols Π which follow, the protocols proceed in super rounds, where in each super round $s = 1, 2, \dots$ the parties invoke a sub-protocol Π^s , wait for its outputs, and then run the next super round. In the first super round, we assume that each P_i has an input (x_i, π_i) , where $J_{\text{In}}(P_i, x_i, \pi_i) = \top$ and that this is the input to the first sub-protocol Π^1 . As a sentinel, let $x_i^0 = x_i$ and $\pi_i^0 = \pi_i$ and define $\Pi^0.J_{\text{Out}} := \Pi.J_{\text{In}}$. Consider then a super round s where in the previous $s - 1$ super rounds the parties ran protocols $\Pi^1, \dots, \Pi^{s-1}$ and in super round s are to run Π^s . For $k = 1, \dots, s - 1$, let y_i^k be the output of P_i from Π^k and let π_i^k be the justifier. Then the message to be input to Π^s by P_i will be computed using a function

$$x_i^s = \text{NxtInp}(\{y_i^k\}_{k=0}^{s-1}) \tag{5}$$

and the accompanying justifier computed using a function

$$\pi_i^s = \text{NxtJst}(\{(y_i^k, \pi_i^k)\}_{k=0}^{s-1}) . \tag{6}$$

Definition 13 (Generic Justifier (Predicate)). *When we say that we use a generic justifier in a setting as described above then we mean that*

$$\text{NxtJst}(\{(y_i^k, \pi_i^k)\}_{k=0}^{s-1}) = \{(y_i^k, \pi_i^k)\}_{k=0}^{s-1} .$$

We call input justifier predicate $\Pi^s.J_{\text{In}}$ generic if it is of the form

$$\begin{aligned} \Pi^s.J_{\text{In}}(\mathbf{P}_j, x_i^s, \{(y_i^k, \pi_i^k)\}_{k=0}^{s-1}) &\equiv \\ x_i^s = \text{NxtInp}(\{y_i^k\}_{k=0}^{s-1}) &\wedge \bigwedge_{k=0}^{s-1} \Pi^k.J_{\text{Out}}(\mathbf{P}_j, y_i^k, \pi_i^k) . \end{aligned}$$

We call $\Pi^s.J_{\text{In}}$ 1-STMA generic iff it is generic and Π only sends messages as part of invoking an instance of STMA as a subprotocol. We call $\Pi^s.J_{\text{In}}$ $(c+1)$ -STMA generic iff it is c -STMA generic or it is generic and Π only sends messages as part of invoking an instance of a c -STMA generic subprotocol. We call $\Pi^s.J_{\text{In}}$ STMA generic iff it is c -STMA generic for some $c \in \mathbb{N}_+$.

We also use generic justifiers for the outputs. We simply show that the output can be computed from justified outputs of the sub-protocols.

Definition 14 (Generic Justifier for Output). In a protocol with σ super-rounds, we give generic justified outputs by computing $x_i^{\sigma+1}$ as if it was an input for a virtual round $\sigma+1$ (which we can also think of as the first round of the next protocol where $x_i^{\sigma+1}$ is input) and then we let the output of $\mathbf{P}_i$ be $y_i = x_i^{\sigma+1}$ and $\pi_i = \pi_i^{\sigma+1}$ where $x_i^{\sigma+1}$ is computed as in Eq. (5) and π_i as in Eq. (6).

Lemma 13. If J is an STMA-generic justifier, then J is an STAM justifier.

Proof. Let $\pi = \{(y_i^k, \pi_i^k)\}_{k=0}^{s-1} \subset S \sqsubset S'$. By Lemma 11 compressing and then decompressing, $\pi' = \text{dec}(\text{enc}(\pi, S), S')$, does not change the value of any y_i^k . Therefore the validity of $x_i^s = \text{NxtInp}(\{y_i^k\}_{k=0}^{s-1})$ does not depend on whether $(\Pi^k, y_i^k, \pi_i^k) \in S$ or $(\Pi^k, y_i^k, \pi_i^{k'}) \in S'$ are used. Assume then that J is simple STMA-generic, i.e., all Π^k are STMA instances. Since S' is a set of justified STMA outputs, $(\Pi^k, y_i^k, \pi_i^{k'}) \in S'$ implies that $\Pi^k.J_{\text{Out}}(\Pi^k, y_i^k, \pi_i^{k'}) = \top$. Therefore $\bigwedge_{k=0}^{s-1} \Pi^k.J_{\text{Out}}(\mathbf{P}_j, y_i^k, \pi_i^{k'}) = \top$. Similarly, if J is STMA-generic then all Π^k are STMA instances of are generic-STMA. If Π^k is an STMA we conclude as above. If Π^k is generic-STMA we recursively conclude from $\Pi^k.J_{\text{Out}}(\mathbf{P}_j, y_i^k, \pi_i^k) = \top$ that $\Pi^k.J_{\text{Out}}(\mathbf{P}_j, , \text{dec}(\text{enc}(\pi_i^k, S), S')) = \top$. Again we get that $\bigwedge_{k=0}^{s-1} \Pi^k.J_{\text{Out}}(\mathbf{P}_j, y_i^k, \pi_i^{k'}) = \top$. $\square$

When an output is generically justified, then one can extract a view of the protocol execution of the party producing it. We capture this in the following definition.

Definition 15 (Weak Unfolded View). Consider an AJO y in some σ -super-round protocol Π as described above with the generic output justifier predicate J_{Out} . Recall that this means that there exists an honest party $\mathbf{P}_i$ that accepts y as output from Π with some justifier π' , i.e., $\Pi.J_{\text{Out}}(\mathbf{P}_i, y, \pi') = \top$. We define the weak unfolded view of y as

$$\text{unfold}(y, \pi') := ((x, \pi), (y^1, \pi^1) \dots, (y^\sigma, \pi^\sigma), y),$$

where π' corresponds to the format prescribed by the generic output justifier predicate J_{Out} . That is, π' contains $x = x^0$ and $\pi = \pi^0$ such that $\Pi.J_{\text{In}}(\mathbf{P}_i, x, \pi) = \top$ (for $\mathbf{P}_i$ as above⁴) and

⁴ Note that by transferability of π , the index i of this party does not matter.

outputs $y^1, \dots, y^\sigma$ of its sub-protocols $\Pi^1, \dots, \Pi^\sigma$ along with justifiers $\pi^1, \dots, \pi^\sigma$ such that $\Pi^s.J_{\text{Out}}(\mathbf{P}_i, y^s, \pi^s) = \top$ for all $1 \leq s \leq \sigma$ and such that $y = \text{NxtInp}(\{y^k\}_{k=0}^\sigma)$ ⁵ is a correctly constructed output.

Remark 7 (Inconsistent Unfolded Views). Note that the weak unfolded view does not demonstrate that the input x^s to Π^s was computed according to the protocol from $(y^0, \dots, y^{s-1})$. It only shows that the output y^s of Π^s , which is included in the justifier π^s , was justified and that the final y was computed from x and these justified y^s . In particular, if we were to unfold the output y^s of Π^s using π^s it might give result in an input $x^{s'}$ to Π^s , where $x^{s'} \neq \text{NxtInp}(\{y_i^k\}_{k=0}^{s-1})$. This is intended and seems crucial in controlling that the size of generic justifiers does not grow exponentially. Namely, because of compression and decompression, the justifier π^s that the receiver of y^s uses to unfold the view might not be the same as the one used by the sender of y^s . It is not only for compression-decompression that it is feature that y^s might not be consistent with $x^s = \text{NxtInp}(\{y^k\}_{k=0}^{s-1})$ in the view of $\mathbf{P}_i$. This will soon allow us that $\mathbf{P}_i$ takes over a justified output y_j^s from another party $\mathbf{P}_j$ for some sub-protocol, i.e., it lets $\mathbf{P}_i$ use $y_i^s = y_j^s$ without having to recursively check consistency of the output y_j^s which it takes over with its own overall execution. It takes over y_j^s if and only if it can be justifier as an output. To remind ourselves that unfolded views may not be globally consistent we call them *weak*.

Definition 16 (Amortised STMA Complexity). *Below when we calculate the communication complexity of protocol we count the total length of inputs the STMA instances at a single party along with the length of the justifiers after compression and after removing all accusations of polarisers in justifiers. We call the sum the amortised STMA complexity.*

The following is a corollary to Lemma 12.

Lemma 14 (Amortised STMA Complexity). *If the amortised STMA complexity of a protocol is ℓ then the communication complexity is $\mathcal{O}(n^3\ell + n^4\lambda)$.*

7 Justified Grade Cast

In STMA, some parties might have output NoMsg while some have $m \neq \text{NoMsg}$. We now show how to upgrade an STMA to a graded STM where the output contains a grade $g \in \{0, 1, 2\}$ which indicates the confidence in this output. When $g = 2$, no AJO can have $m \neq \text{NoMsg}$. Furthermore, grades are at most 1 apart and honest senders always produce grade 2. A detailed definition is given in Definition 17. Our protocol is given in Fig. 6.

Definition 17 (Graded Send Transferable Message). *Let Π be a protocol run among n parties $\mathbf{P}_1, \dots, \mathbf{P}_n$ and let J_{Msg} be a parametrisable input justifier predicate. Assume that Π specifies a designated sender $\mathbf{P}_s$ holding input (m, π_{Msg}) such that $J_{\text{Msg}}(\mathbf{P}_s, m, \pi_{\text{Msg}}) = \top$ and parties terminate upon generating output in Π . As part of its code the protocol specifies an output justifier predicate $\Pi.J_{\text{Out}}$. We say that Π is a graded send transferable message (GSTM) protocol if it has the following properties:*

⁵ Recall that by Definition 14 we compute the output as if it was the input for a next virtual round.

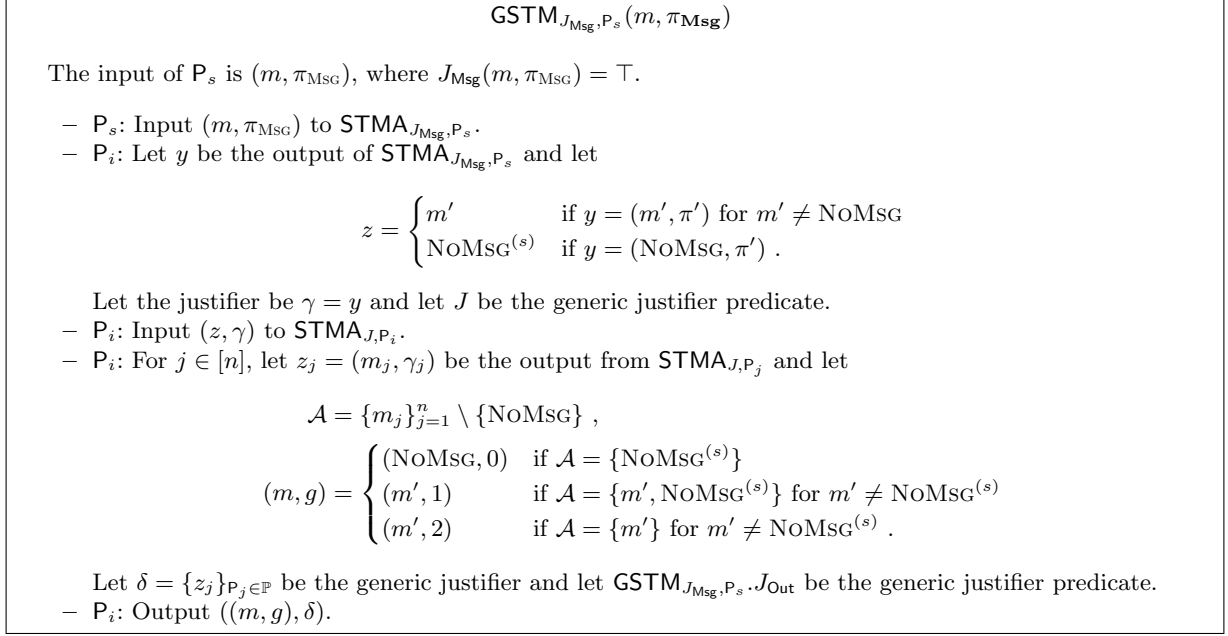


Fig. 6. A justified gradecast protocol.

- **Correctness:** Honest P_i outputs $y_i = ((m_i, g_i), \pi_i)$ for $g_i \in \{1, 2\}$ and $m_i \neq \text{NoMsg}$ or $y_i = ((m_i = \text{NoMsg}, g_i = 0), \pi_i)$.
- **Justified Output:** When honest P_i outputs $y_i = ((m_i, g_i), \pi_i)$ then $J_{\text{Out}}(P_i, (m_i, g_i), \pi_i) = \top$.
- **Justified Message:** For all AJOs $y_i = ((m_i, g_i), \pi_i)$ either $m_i = \text{NoMsg}$ and $g_i = 0$ or π_i is of the form $\pi_i = (\pi_i^1, \pi_i^2)$ and $J_{\text{Msg}}(m_i, \pi_i^1) = \top$.
- **Validity:** If P_s is honest and has input (m, π_{Msg}) , then all honest parties P_j have output $m_j = m$ and $g_j = 2$. Furthermore, for all AJOs (m', g') it holds that $m' = m$ and $g' = 2$.
- **Graded Agreement:** For all AJOs (m^1, g^1) and (m^2, g^2) it holds that $|g^1 - g^2| \leq 1$ and if $g^1, g^2 > 0$ then $m^1 = m^2$.

Theorem 3. Protocol $\text{GSTM}_{J_{\text{Msg}}, P_s}$ is a GSTM for $t < n$. Assume that GSTM has inputs (m, π) with $|m| \leq \ell$. Then the protocol has STMA communication complexity $\mathcal{O}(n\ell)$.

As for STMA, all properties but validity and agreement are trivial and so is the communication complexity.

Lemma 15 (Validity). Protocol $\text{GSTM}_{J_{\text{Msg}}, P_s}$ in Figure 6 is valid.

Proof. Suppose that P_s is honest and has input m . By validity of $\text{STMA}_{J_{\text{Msg}}, P_s}$, every AJO y for that protocol is $y = (m', \pi')$ for $m' = m$. Now let $((m, g), \delta)$ be any AJO of $\text{GSTM}_{J_{\text{Msg}}, P_s}(m, \pi_{\text{Msg}})$ and assume for the sake of contradiction that $g \in \{0, 1\}$. Then the weak unfolded view of $((m, g), \delta)$ contains an AJO for $(\text{NoMsg}, 0)$ or $(m', 1)$ from STMA_{J, P_j} . By definition of $\mathcal{A}$ the weak unfolded view of either of these AJOs will contain an AJO for $y = (\text{NoMsg}, \pi')$ from $\text{STMA}_{J_{\text{Msg}}, P_s}$, a contradiction. $\square$

Lemma 16 (Graded Agreement). *Protocol $\text{GSTM}_{J_{\text{Msg}}, P_s}$ in Figure 6 has graded agreement.*

Proof. Suppose we have AJOs $((m_i, g_i), \delta_i)$ and $((m_j, g_j), \delta_j)$ with $g_i, g_j > 0$. By the properties of the generic justifier, it must be the case that the weak unfolded view of $((m_i, g_i), \delta_i)$ holds an AJO for $m_i \neq \text{NoMsg}$ from some STMA_{J, P_k} . By the justified message property of STMA it follows that m_i is justified using the input justifier J of STMA_{J, P_k} , which was the generic justifier checking that m_i was an AJO of $\text{STMA}_{J_{\text{Msg}}, P_s}$. Symmetrically, we can conclude that m_j was an AJO of $\text{STMA}_{J_{\text{Msg}}, P_s}$. By the agreement property of $\text{STMA}_{J_{\text{Msg}}, P_s}$ it follows that $m_i = m_j$. It remains to show that it cannot be the case for two AJOs that $g_i = 2$ and $g_j = 0$. Assume that $g_i = 0$. Suppose then for the sake of contradiction that we have AJOs $((m_i, g_i), \delta_i)$ and $((m_j, g_j), \delta_j)$ with $g_i = 0$ and $g_j = 2$. The weak unfolded view of $((m_i, g_i), \delta_i)$ holds an AJO for $m_i = \text{NoMsg}^{(s)}$ from all STMA_{J, P_k} which did not output NoMsg , in particular for all honest P_k . Symmetrically, we can conclude that m_j was an AJO from STMA_{J, P_k} for all honest P_j . Since $g_j = 2$ we have $m_j \neq \text{NoMsg}^{(s)}$. Since there is at least one honest P_j we have found one STMA_{J, P_k} where P_k is honest and it has AJO $\text{NoMsg}^{(s)}$ and AJO $m_j \neq \text{NoMsg}^{(s)}$. This breaks validity of STMA_{J, P_k} as the honest P_k cannot have inputted both $\text{NoMsg}^{(s)}$ and $\neq \text{NoMsg}^{(s)}$. $\square$

8 Early Stopping Broadcast

We now present our main result, early stopping broadcast.

8.1 Diagonal Cast

We begin by building an early stopping protocol, called DC (for *diagonal cast*), which is early stopping, but may run up to $O(t^2)$ rounds. The protocol will be a justifiable broadcast in the sense of Definition 6, which implies that it is also a broadcast protocol. The protocol DC uses GSTM as (blackbox) sub-protocol. Since we use DC as a building block in our final protocol, we also make its output justified. During the protocol we use a helper function computing a party's next vote. In each round the parties run one GSTM to get output (m, g) . Consider a party which in the previous rounds saw outputs $(m^1, g^1), \dots, (m^r, g^r)$. Then it should use the m from the latest GSTM with $g^\rho > 0$. If no such ρ exists it should use $\text{NoMsg}^{(s)}$ to indicate the sender P_s was corrupt. More formally we use this function:

$$\text{NxtInp}((m^1, g^1), \dots, (m^r, g^r)) = \begin{cases} \text{NoMsg}^{(s)} & \text{if } g^1 = \dots = g^r = 0 \\ m_{\max\{i \in [r] \mid g_i > 0\}} & \text{otherwise.} \end{cases}$$

Below we let $i^* = \max\{i \in [r] \mid g_i > 0\}$ when we are not in the case $g^1 = \dots = g^r = 0$. At the end of the protocol we want to map $\text{NoMsg}^{(s)}$ to NoMsg . For this we use a simple helper: $\text{Out}(m) = \text{NoMsg}$ if $m = \text{NoMsg}^{(s)}$ and $\text{Out}(m) = m$ otherwise. The protocol is given in Fig. 7.

Lemma 17 (Validity). *DC satisfies validity.*

Diagonal Cast $\text{DC}_{J_{\text{Msg}}, P_s}$

Without loss of generality, assume that P_1 is the sender, i.e., $s = 1$.

- In round 1:
 - P_1 : Has input (m, π_{Msg}) with $J_{\text{Msg}}(P_1, m, \pi_{\text{Msg}}) = \top$ and inputs it to $\text{GSTM}_{J_{\text{Msg}}, P_1}$.
 - P_i : Let $y = ((m_i^1, g_i^1), \pi_i^1)$ be the output from $\text{GSTM}_{J_{\text{Msg}}, P_1}$.
 - P_i : If $g_i^1 = 2$ then send $(1, (m_i^1, g_i^1), \pi_i^1)$ to all other parties, output $m = \text{Out}(m_i^1)$, run for one more round (to ensure that no honest party terminates until all honest gave output), and then terminate.
- For $j \in \{2, \dots, n\}$ do as follows:
 - P_j : Compute $m_j = \text{NxtInp}((m_j^1, g_j^1), \dots, (m_j^{j-1}, g_j^{j-1}))$.
 - P_j : Compute the generic justifier $\gamma_j = \{((m_j^k, g_j^k), \pi_j^k)\}_{k=1}^{j-1}$ and let J^j be the generic justifier predicate for this round.
 - P_j : Input (m_j, γ_j) to GSTM_{J^j, P_j} .
 - P_i : Let $y = ((m_i^j, g_i^j), \pi_i^j)$ be the output from GSTM_{J^j, P_j} .
 - P_i : If $g_i^j = 2$ then send $(j, (m_i^j, g_i^j), \pi_i^j)$ to all other parties, output $m = \text{Out}(m_i^j)$, run for one more round, and then terminate.
- P_i : Upon receiving a tuple $(j, (m', g'), \pi')$ (for any $j \in [n]$ and in any round) such that $\text{GSTM}_{J^j, P_j} \cdot J_{\text{Out}}(P_i, (m', g'), \pi') = \top$ and $g' = 2$, forward it to all parties, output $m = \text{Out}(m')$, run for one more round, and then terminate.
- The output justifier for the protocol is $\text{DC}_{J_{\text{Msg}}, P_s} \cdot J_{\text{Out}}(m, \pi)$, which is true if $\pi = (j, (m', g'), \pi')$ and $j \in [n]$ and $\text{GSTM}_{J^j, P_j} \cdot J_{\text{Out}}(P_i, (m', g'), \pi') = \top$ and $m = \text{Out}(m')$.

Fig. 7. The broadcast protocol.

Proof. Suppose that P_1 is an honest sender holding input (m, π_{Msg}) . Since $J_{\text{Msg}}(P_1, m, \pi_{\text{Msg}}) = \top$ validity of $\text{GSTM}_{J_{\text{Msg}}, P_1}$ implies that all honest parties get output $(m, 2, \pi)$. Hence, all parties terminate with output m after running for one round. $\square$

Lemma 18 (Stabilisation). *Let j be the minimal iteration such that for some honest P_i an AJO $((m, 2), \pi)$ satisfying $\text{GSTM}_{J^j, P_j} \cdot J_{\text{Out}}(P_i, (m, 2), \pi) = \top$ can be produced. Then for any $j' \geq j$ and any honest $P_{i'}$ and any AJO $((m', 2), \pi')$ satisfying $\text{GSTM}_{J^{j'}, P_{j'}} \cdot J_{\text{Out}}(P_{i'}, (m', g'), \pi') = \top$ it holds that $g' = 0$ or $m' = m$. Furthermore, if $j' = j$ then $g' > 0$ and thus $m' = m$.*

Proof. We do induction in $|j' - j| = 0, 1, \dots$. For the base case $|j' - j| = 0$ we have $j' = j$ and that $g' \in \{1, 2\}$ and $m' = m$ by graded agreement of GSTM_{J^j, P_j} . Assume then the induction hypothesis for all $|j' - j| < \ell$. We prove it for $|j' - j| = \ell$. We have to prove that $g' = 0$ or $m = m'$. So it is enough to prove that if $g' > 0$ then $m' = m$. When $g' > 0$ then by the justified message property of $\text{GSTM}_{J^{j'}, P_{j'}}$ we have that m' is justified by $J^{j'}$, which was the generic justifier for iteration j' . Therefore m' was computed as

$$m' = \text{NxtInp}((m^1, g^1), \dots, (m^{j'-1}, g^{j'-1}))$$

from AJOs. By induction hypothesis $g^j \geq 1$ and $m^j = m$, and for $j \leq k \leq j' - 1$ it holds that $g^k = 0$ or $m^k = m$. This by construction gives

$$\text{NxtInp}((m^1, g^1), \dots, (m^{j'-1}, g^{j'-1})) = m,$$

as desired. $\square$

Corollary 3. *DC has agreement.*

Proof. When an honest party P_i produces output m' then it by construction produces or receives an AJO $(m', g = 2)$ for some $\text{GSTM}_{J_{j'}, P_{j'}}$. Clearly there is a smallest j for which an AJO $(m, g = 2)$ can be produced for GSTM_{J_j, P_j} . By Lemma 18 it holds that $m' = m$. This holds for all honest outputs m' . $\square$

Lemma 19. *DC terminates in $8(f + 1)(f + 2)$ rounds. If P_1 is honest then it runs in at most $8(f + 2)$ rounds.*

Proof. The protocol terminates at the latest once the first honest party acts as the sender in some iteration j as this gives $g_j = 2$. This is worstcase in iteration $(f + 1)$. Each iteration runs one GSTM , which uses two STMA , which each uses two STM , which each uses $f + 2$ rounds. This gives $4(f + 2)$ rounds per GSTM , and applying the staggering compiler contributes a factor of 2. $\square$

Lemma 20. *When DC is run on an input of size ℓ then it has STMA complexity $\mathcal{O}(f^2 n \ell)$.*

Proof. The protocol calls $\mathcal{O}(f^2)$ instances of GSTM , one in each round. Each GSTM is called with an input of size $\ell = \ell$, so each GSTM has STMA complexity $n \ell$. This gives the desired bound. $\square$

8.2 Weak Early Stopping $\mathcal{O}(f)$ and Worstcase $\mathcal{O}(t)$

The protocol DC can do early stopping, but if there are $f = \omega(\sqrt{t})$ corruptions it runs for more than $\mathcal{O}(t)$ rounds, which is asymptotically sub-optimal. We solve this by capping the running time at $\mathcal{O}(t)$. Doing this safely is subtle, and we now present a protocol with weak early stopping which helps doing it safely. Weak early stopping means that the protocol achieves early stopping when the sender is honest. If the sender is corrupt it may run for $\mathcal{O}(n)$ rounds. We describe the protocol with P_1 as sender, but it can be adopted to any P_s .

$\text{WES}_{J_{\text{Msg}}, P_1}$	
–	P_1 : Has input (m, π_{Msg}) with $J_{\text{Msg}}(P_1, m, \pi_{\text{Msg}}) = \top$ and inputs it to $\text{GSTM}_{J_{\text{Msg}}, P_1}$.
–	P_1 : Let $((m, g), \pi)$ be the output of $\text{GSTM}_{J_{\text{Msg}}, P_1}$ and run $\text{DSC}_{P_1, J}$ with input $((m, g), \pi)$ by P_1 , where J is the generic justifier predicate.
–	P_i : Upon receiving a tuple $((m', g'), \pi')$ as output from $\text{GSTM}_{J_{\text{Msg}}, P_1}$ or relayed by another party, where $\text{GSTM}_{J_{\text{Msg}}, P_1} \cdot J_{\text{Out}}(P_i, (m', g'), \pi') = \top$ and $g' = 2$, forward it to all parties, output $m = m'$, and terminate.
–	If $\text{DSC}_{P_1, J_{\text{Msg}}}$ outputs $((m, g), \pi)$ before an output was produced using the above rule, output as follows. If $\text{GSTM}_{J_{\text{Msg}}, P_1} \cdot J_{\text{Out}}(P_i, (m, g), \pi) = \top$ and $g > 0$, output m . Otherwise, output NoMsg.

Fig. 8. The weak early stopping broadcast protocol.

Theorem 4. *The protocol $\text{WES}_{J_{\text{Msg}}, P_1}$ is a broadcast protocol. Parties terminate one round apart and the round complexity is $\mathcal{O}(t)$. If P_1 is honest then the round complexity is $\mathcal{O}(f)$. The GSTM complexity is $\mathcal{O}(\ell)$ and there is an additional communication of $\mathcal{O}(n^2 \ell + n^3 \lambda)$ from DSC.*

Proof. Validity is trivial: if P_1 is honest then $\text{GSTM}_{J_{\text{Msg}}, P_1}$ outputs $((m, g), \pi)$ with $g = 2$ and all honest parties output m . This happens within one run of GSTM , so in $\mathcal{O}(f)$ rounds. We argue agreement. If all honest parties give output by receiving a tuple $((m', g'), \pi')$ which is an AJO for $\text{GSTM}_{J_{\text{Msg}}, P_1}$ and with $g' = 2$ then agreement follows from agreement of $\text{GSTM}_{J_{\text{Msg}}, P_1}$. If all honest parties give output by receiving $((m, g), \pi)$ from DSC then agreement follows from agreement of DSC . Assume then that some honest P_i gives output using $((m' = m_i, g' = 2), \pi')$ which is an AJO for $\text{GSTM}_{J_{\text{Msg}}, P_1}$ and some honest P_j gives output m_j by receiving $((m, g), \pi)$ from $\text{DSC}_{P_1, J}$. Since $g' = 2$ and $m' = m_i$ it follows from graded agreement for $\text{GSTM}_{J_{\text{Msg}}, P_1}$ that $m = m_i$ and $g > 0$ for all AJOs for $\text{GSTM}_{J_{\text{Msg}}, P_1}$. Since J is the generic justifier it follows that all AJOs $((m, g), \pi)$ for $\text{DSC}_{P_1, J}$ has $m'' = m_i$ and $g'' > 0$. Therefore P_j has output $m_j = m = m_i$. Parties terminate one round apart as they terminate one round apart in DSC and if they terminate by the $g' = 2$ rule then they forward the AJO and then all honest parties terminate in the next round. $\square$

8.3 Early Stopping $\mathcal{O}(f^2)$ and Worstcase $\mathcal{O}(t)$

We now give a broadcast protocol Capped Diagonal Cast which basically runs Diagonal Cast for a capped number of rounds and uses WES to have all parties report whether they saw an output from DC before the time cap. Since the reports are sent with WES parties will agree on the reports and make the same decision. Furthermore, if an honest party saw an output it will be reported with early stopping. We again describe the protocol with P_1 as sender, but can trivially adopt to any P_s .

Capped Diagonal Cast $\text{CDC}_{P_1, J_{\text{Msg}}}$

We describe the protocol from the view of party P_i .

- P_1 : Has input (m, π_{Msg}) with $J_{\text{Msg}}(m, \pi_{\text{Msg}}) = \top$ and inputs it to $\text{DC}_{P_1, J_{\text{Msg}}}$.
- P_i : Participate in $\text{DC}_{P_1, J_{\text{Msg}}}$ for at most $8(t+1)$ rounds.^a
- P_i : If $\text{DC}_{P_1, J_{\text{Msg}}}$ produced output (m, π) in or before round $8(t+1)$ then run $\text{WES}_{P_i, \top}$ with input (m, π) in the round where $\text{DC}_{P_1, J_{\text{Msg}}}$ produced output. If by round $8(t+1)$ protocol $\text{DC}_{P_1, J_{\text{Msg}}}$ did not produce output run $\text{WES}_{P_i, \top}$ with input NoMsg in round $8(t+1)$.
- P_i : If and when the first $\text{WES}_{P_j, \top}$ outputs (m, π) such that $\text{DC}_{P_1, J_{\text{Msg}}} \cdot J_{\text{Out}}(m, \pi) = \top$, output m .
- P_i : If $\text{WES}_{P_1, \top}, \dots, \text{WES}_{P_n, \top}$ all terminated and none had an output (m, π) such that $\text{DC}_{P_1, J_{\text{Msg}}} \cdot J_{\text{Out}}(m, \pi) = \top$, then output NoMsg .

^a Here we count base communication rounds, not iterations of GSTM .

Fig. 9. An Early Stopping Broadcast protocol with $\mathcal{O}(\min((f+1)^2, n))$ rounds.

Theorem 5. *The protocol $\text{CDC}_{J_{\text{Msg}}, P_1}$ is a broadcast protocol. Parties terminate one round apart and the round complexity is $\mathcal{O}(\min(f^2, t))$. If the sender is honest output is given in $\mathcal{O}(f)$ rounds. From $\text{CDC}_{J_{\text{Msg}}, P_1}$ we can get a broadcast protocol EBC for definition Definition 1 with the same round complexity simply by dropping the input justifier predicate J_{Msg} and the justifier π_{Msg} . The communication complexity is $\mathcal{O}(n^4(\min(f^2, t)\ell + \lambda))$.*

Proof. First note that all $\text{WES}_{P_i, \top}$ are started at most one round apart. Namely, no party starts them later than by round $8(t+1)$. So if they are to be started 2 rounds apart the first is started in round $8(t+1) - 2$ or earlier. But then $\text{DC}_{P_1, J_{\text{Msg}}}$ terminated by round $8(t+1) - 2$ at the first honest party. But then it terminated by round $8(t+1) - 1$ at all honest. Hence all honest started $\text{WES}_{P_i, \top}$ exactly when $\text{DC}_{P_1, J_{\text{Msg}}}$ terminated, which is at most one round apart. We can then apply the staggering compiler to ensure that $\text{WES}_{P_i, \top}$ tolerates being started one round apart. This gives a $\mathcal{O}(1)$ blowup in round complexity. We then argue validity: if P_1 is honest then $\text{DC}_{J_{\text{Msg}}, P_1}$ outputs (m, π) within $8(f+2) \leq 8(t+2)$ rounds. Therefore an honest party P_j will send (m, π) on $\text{WES}_{P_j, \top}$ which will output (m, π) within $\mathcal{O}(f)$ rounds and then all honest output m . This all happens within $\mathcal{O}(f)$ rounds. Agreement is trivial from agreement of WES , as the output is computed deterministically from the outputs of $\text{WES}_{P_1, \top}, \dots, \text{WES}_{P_n, \top}$. Consider then the round complexity. We have that all copies of WES are started after $\mathcal{O}(\min(f^2, t))$ rounds as $\text{DC}_{P_1, J_{\text{Msg}}}$ stops after $\mathcal{O}(f^2)$ rounds and we cap after $8(t+1)$ rounds. If $\text{DC}_{P_1, J_{\text{Msg}}}$ did give an output at all honest parties before round $8(t+1)$ then it will be input to $\text{WES}_{P_i, \top}$ which will output after $\mathcal{O}(f)$ rounds, and hence output is produced in $\mathcal{O}(f^2)$ rounds as $\text{DC}_{P_1, J_{\text{Msg}}}$ terminates in $\mathcal{O}(f^2)$ rounds. If $\text{DC}_{P_1, J_{\text{Msg}}}$ did not give output within $8(t+1)$ rounds then $8(t+1) = \mathcal{O}(f^2)$, so $\mathcal{O}(\min(f^2, t)) = \mathcal{O}(t)$. And in this case the overall protocols runs for at most $\mathcal{O}(t)$ rounds as each WES terminates in $\mathcal{O}(t)$ rounds.

Running DC capped for $8(t+1)$ rounds gives STMA complexity $\mathcal{O}(\min(f^2, t)n\ell)$. This results in communication complexity $\mathcal{O}(\min(f^2, t)n^4\ell + n^4\lambda)$. Running the n instances of WES gives STMA complexity $\mathcal{O}(n\ell)$ and there is an additional communication of $\mathcal{O}(n^3\ell + n^4\lambda)$, which is dominated by the above. $\square$

9 Acknowledgements

We would like to thank Juan Garay for fruitful discussions in the early stages of this work. We would also like to thank Fatima Elsheimy for many detailed technical and editorial comments.

References

1. Ittai Abraham, T.-H. Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Communication complexity of byzantine agreement, revisited. In Peter Robinson and Faith Ellen, editors, *38th ACM PODC*, pages 317–326. ACM, July / August 2019.
2. Ittai Abraham and Danny Dolev. Byzantine agreement with optimal early stopping, optimal resilience and polynomial complexity. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 605–614. ACM Press, June 2015.
3. Timoth   Albouy, Davide Frey, Michel Raynal, and Fran  ois Ta  iani. Good-case early-stopping latency of synchronous byzantine reliable broadcast: The deterministic case. In Christian Scheideler, editor, *36th International Symposium on Distributed Computing, DISC 2022, October 25-27, 2022, Augusta, Georgia, USA*, volume 246 of *LIPICs*, pages 4:1–4:22. Schloss Dagstuhl - Leibniz-Zentrum f  r Informatik, 2022.
4. Andreea B. Alexandru, Julian Loss, Charalampos Papamanthou, and Giorgos Tsimos. Sublinear-round broadcast without trusted setup against dishonest majority. Cryptology ePrint Archive, Report 2022/1383, 2022. <https://eprint.iacr.org/2022/1383>.

5. Piotr Berman and Juan A. Garay. $n/4$ -resilient distributed consensus in $t + 1$ rounds. *Mathematical Systems Theory*, 26(1):3–19, 1993.
6. Piotr Berman, Juan A. Garay, and Kenneth J. Perry. Towards optimal distributed consensus (extended abstract). In *30th FOCS*, pages 410–415. IEEE Computer Society Press, October / November 1989.
7. Piotr Berman, Juan A. Garay, and Kenneth J. Perry. Optimal early stopping in distributed consensus (extended abstract). In *WDAG*, pages 221–237, 1992.
8. T.-H. Hubert Chan, Rafael Pass, and Elaine Shi. Sublinear-round byzantine agreement under corrupt majority. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 246–265. Springer, Heidelberg, May 2020.
9. Tushar Deepak Chandra and Sam Toueg. Time and message efficient reliable broadcasts. In *WDAG*, pages 289–303, 1990.
10. Danny Dolev and Christoph Lenzen. Early-deciding consensus is expensive. In Panagiota Fatourou and Gadi Taubenfeld, editors, *32nd ACM PODC*, pages 270–279. ACM, July 2013.
11. Danny Dolev and Rüdiger Reischuk. Bounds on information exchange for byzantine agreement. In Robert L. Probert, Michael J. Fischer, and Nicola Santoro, editors, *1st ACM PODC*, pages 132–140. ACM, August 1982.
12. Danny Dolev, Rüdiger Reischuk, and H. Raymond Strong. Early stopping in byzantine agreement. *J. ACM*, 37(4):720–741, 1990.
13. Danny Dolev and H. Raymond Strong. Authenticated algorithms for Byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.
14. Danny Dolev and Andrew Chi-Chih Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–207, 1983.
15. Paul Feldman and Silvio Micali. Optimal algorithms for byzantine agreement. In *20th ACM STOC*, pages 148–161. ACM Press, May 1988.
16. Matthias Fitzi and Jesper Buus Nielsen. On the number of synchronous rounds sufficient for authenticated byzantine agreement. In *DISC*, pages 449–463, 2009.
17. Juan A. Garay, Jonathan Katz, Chiu-Yuen Koo, and Rafail Ostrovsky. Round complexity of authenticated broadcast with a dishonest majority. In *48th FOCS*, pages 658–668. IEEE Computer Society Press, October 2007.
18. Juan A. Garay and Yoram Moses. Fully polynomial byzantine agreement in $t+1$ rounds. In *25th ACM STOC*, pages 31–41. ACM Press, May 1993.
19. Juan A. Garay and Yoram Moses. Fully polynomial byzantine agreement for $n \geq 3t$ processors in $t + 1$ rounds. *SIAM Journal on Computing*, 27(1):247–290, 1998.
20. Jonathan Katz and Chiu-Yuen Koo. On expected constant-round protocols for byzantine agreement. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 445–462. Springer, Heidelberg, August 2006.
21. Idit Keidar and Sergio Rajsbaum. A simple proof of the uniform consensus synchronous lower bound. *Information Processing Letters*, 85(1):47–52, 2003.
22. Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Trans. Programming Languages and Systems*, 4(3):382–401, 1982.
23. Christoph Lenzen and Sahar Sheikholeslami. A recursive early-stopping phase king protocol. In *ACM PODC*, pages 60–69. ACM, 2022.
24. Yehuda Lindell, Anna Lysyanskaya, and Tal Rabin. Sequential composition of protocols without simultaneous termination. In Aletta Ricciardi, editor, *21st ACM PODC*, pages 203–212. ACM, July 2002.
25. Silvio Micali. Very simple and efficient byzantine agreement. In Christos H. Papadimitriou, editor, *ITCS 2017*, volume 4266, pages 6:1–6:1, 67, January 2017. LIPIcs.
26. Jesper Buus Nielsen. *On Protocol Security in the Cryptographic Model*. PhD thesis, Aarhus University, 2003.
27. Philippe Raipin Parvédy and Michel Raynal. Optimal early stopping uniform consensus in synchronous systems with process omission failures. In *SPAA*, volume 302–310, 2004.
28. Kenneth J. Perry and Sam Toueg. An authenticated byzantine generals algorithm with early stopping. Technical report, Cornell University, 1984.
29. Shravan Srinivasan, Julian Loss, Giulio Malavolta, Kartik Nayak, Charalampos Papamanthou, and Sri Aravinda Krishnan Thyagarajan. Transparent batchable time-lock puzzles and applications to byzantine consensus. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 554–584. Springer, Heidelberg, May 2023.
30. Jun Wan, Hanshen Xiao, Srinivas Devadas, and Elaine Shi. Round-efficient byzantine broadcast under strongly adaptive and majority corruptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 412–456. Springer, Heidelberg, November 2020.

31. Jun Wan, Hanshen Xiao, Elaine Shi, and Srinivas Devadas. Expected constant round byzantine broadcast under dishonest majority. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 381–411. Springer, Heidelberg, November 2020.